

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ



BÁO CÁO LAB THIẾT KẾ VI MẠCH SỐ
LAB 3: DESIGN OF COMBINATIONAL AND SEQUENTIAL
CIRCUITS

Lớp: L01 - NHÓM: 08

GVHD: Đặng Hoàng Tiền
SV: Vũ Nhật Huy - 2311267
Huỳnh Trung Kiên - 2311729
Trịnh Nguyễn Phước Điền - 2310741

Thành phố Hồ Chí Minh, tháng 4 năm 2026

Mục lục

1. Experiment 1: Design a combinational circuit - A 1-bit Full Adder.	4
1.1 Original 1-bit FA	4
1.1.1 Xác minh thiết kế và đo đặc timing, công suất tiêu thụ	4
1.2 28T Full Adder	16
1.2.1 Xác minh thiết kế và đo đặc timing, công suất tiêu thụ	16
1.3 Question	29
2. Experiment 2: Design a sequential circuit - A PRBS generator.	31
2.1 Schematic và testbench thiết kế bộ PRBS	31
2.2 Thực hiện tính toán sơ bộ để khoanh vùng Fmax	33
2.2.1 Đo lại t_logic_max (IN1, IN2) của cổng XOR2	34
2.2.2 Đo lại t_cq và t_setup của D_FF	35
2.3 Thực hiện đo quét tần số xung quanh F_MAX lý thuyết trên bộ PRBS:	38
2.3.1 Hình ảnh mô phỏng quét tần số từ 10G đến 20G bước nhảy 1G	38
2.3.2 Kết luận về tần số hoạt động tối đa (Fmax) của bộ tạo mã PRBS-4bit	43
3. Experiment 3: Design a VerilogA model.	45

Danh sách hình vẽ

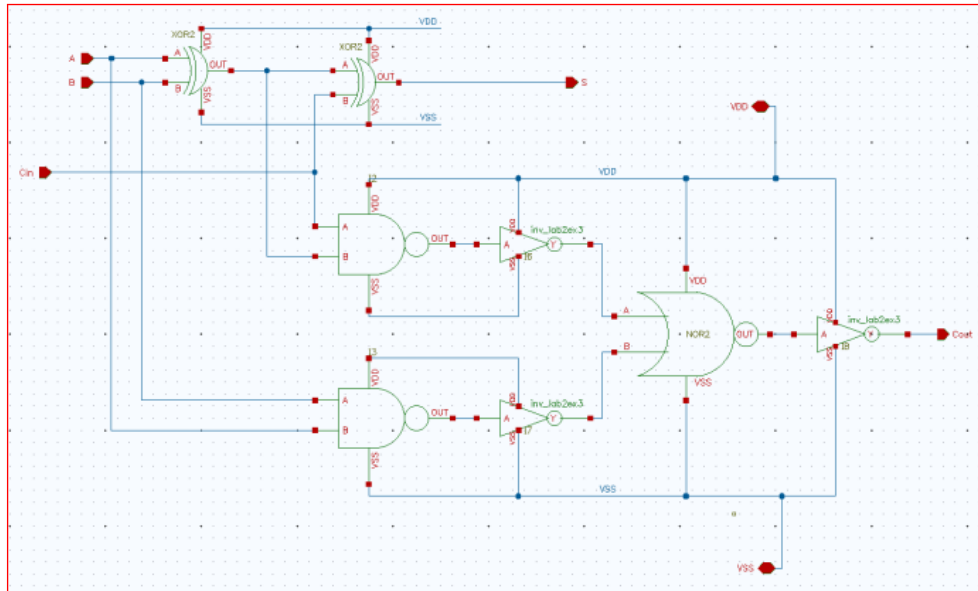
1	Schematic của cổng Full Adder theo mô hình ghép cổng logic (Original FA)	4
2	Mạch testbench	4
3	Dạng sóng transient	5
4	Trạng thái giả tại thời điểm chuyển trạng thái	5
5	Bảng chân trị của cổng 1-bit Full Adder	6
6	Hàm đo được cài đặt để đo các thông số timing và công suất tiêu thụ	6
7	Xác minh các hàm đo	6
8	Critical path của cổng basic 1-bit Full Adder	7
9	Do trễ lan truyền từ A đến ngõ ra S	7
10	Layout của cổng inverter được lấy từ lab2 ex3	8
11	Layout sơ bộ của cổng Original 1-bit FA trước khi đi các đường VDD và VSS	9
12	kết quả khi chạy DRC lần đầu	10
13	Layout sau khi hoàn thành check DRC cho các phần tử logic và tiến hành cấp nguồn cho thiết kế	10
14	kết quả khi hoàn thành DRC	11
15	Via từ Metal1 đến metal4 gây short với metal3 của VDD	11
16	Thiếu Via nối poly với metall	12
17	Các cực G của 2 body chưa được nối với nhau	13
18	Lỗi label nằm ở layer khác	14
19	kết quả sau khi hoàn thành sửa lỗi LVS	14
20	Kết quả layout hoàn thiện của cổng Original Full Adder	15
21	Schematic của cổng 28T Full Adder	16
22	Dạng sóng xác minh thiết kế tại tất cả các trường hợp	16
23	Hàm đo được sử dụng để tìm các thông số timing cùng công suất tiêu thụ	17
24	Critical path của cổng basic 1-bit Full Adder	17
25	các tổ hợp được lựa chọn để cài đặt testbench đo t _{pd}	17
26	kết quả đo t _{pd} của 28T fulladder	18
27	Layout trước khi thực hiện DRC và LVS	19
28	kết quả sau khi hoàn thành sửa lỗi DRC	26
29	Layout sau bước DRC	27
31	Các thành phần phát sinh	30
32	Sơ đồ mạch thiết kế bộ PRBS	31
33	Sơ đồ mạch D_FF sử dụng để thiết kế bộ PRBS	31
34	Sơ đồ mạch cổng XOR2 sử dụng trong thiết kế bộ PRBS	32
35	Testcase đưa vào nguồn vpulse CLK của bộ PRBS	32
36	Chọn điều kiện đầu tổ hợp 1111 để bộ PRBS hoạt động	33
37	Hình minh họa về Concept đường đi dữ liệu	34
38	Do t _{pdf} khi IN1 = 1V, IN2 vs OUT	34
39	Do t _{pdr} khi IN1 = 1V, IN2 vs OUT	35
40	Do t _{pdr} khi IN1 = 0V, IN2 vs OUT	35
41	Do t _{pdf} khi IN1 = 0V, IN2 vs OUT	35
42	Testcase đo chung cho t _{cq} và t _{setup}	36
43	Mô phỏng đúng chức năng và thực hiện đo t _{cq} (thời gian để ngõ ra đáp ứng kể từ xung clock - từ clk đến Q)	36
44	Kết quả đo bằng hàm	36
45	Kết quả đo t _{setup}	37

46	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 10G$	38
47	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 11G$	38
48	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 12G$	39
49	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 13G$	39
50	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 14G$	40
51	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 15G$	40
52	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 16G$	41
53	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 17G$	41
54	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 18G$	42
55	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 19G$	42
56	Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 20G$	43
57	Kết quả chạy code Python bộ tạo mã PRBS-4bit	44
58	Dạng sóng mô phỏng bộ PRBS-4bit	44
59	Code VerilogA khối 4 bit counter-up	45
60	Symbol khối 4 bit counter-up	45
61	Code VerilogA khối 4 bit decoder	46
62	Symbol khối 4 bit decoder	46
63	Mux transmission gate	47
64	Khối verilogA trong mạch testbench	47
65	Khối resistor ladder và mux 16 sang 1 dùng transmission gate	48
66	Kết quả khối 4 bit counter up	49
67	Kết quả khối 4 bit decoder	49
68	Kết quả ngõ ra mux 16 sang 1	50

1. Experiment 1: Design a combinational circuit - A 1-bit Full Adder.

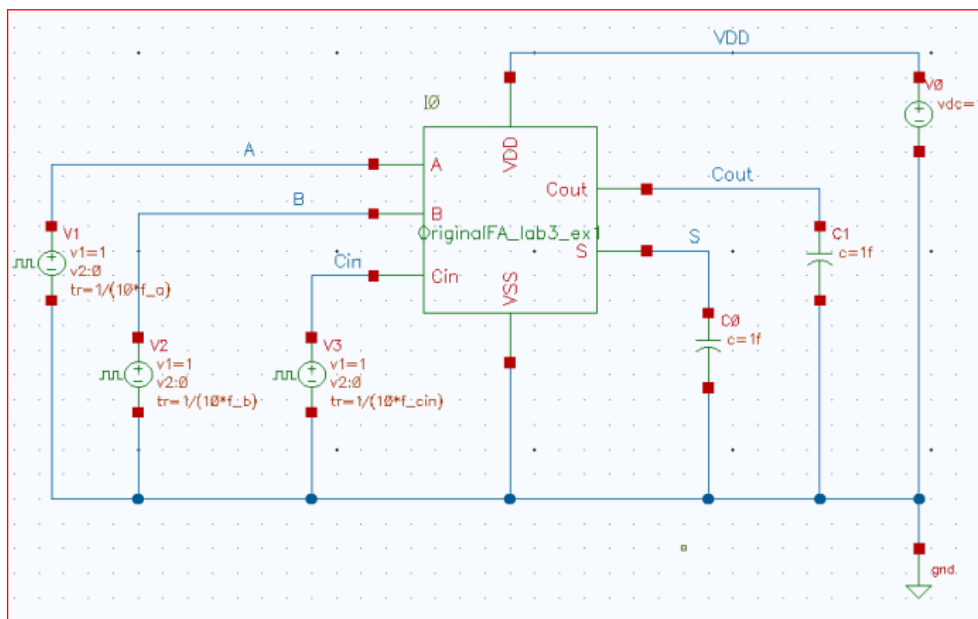
1.1 Original 1-bit FA

1.1.1 Xác minh thiết kế và đo đặc timing, công suất tiêu thụ

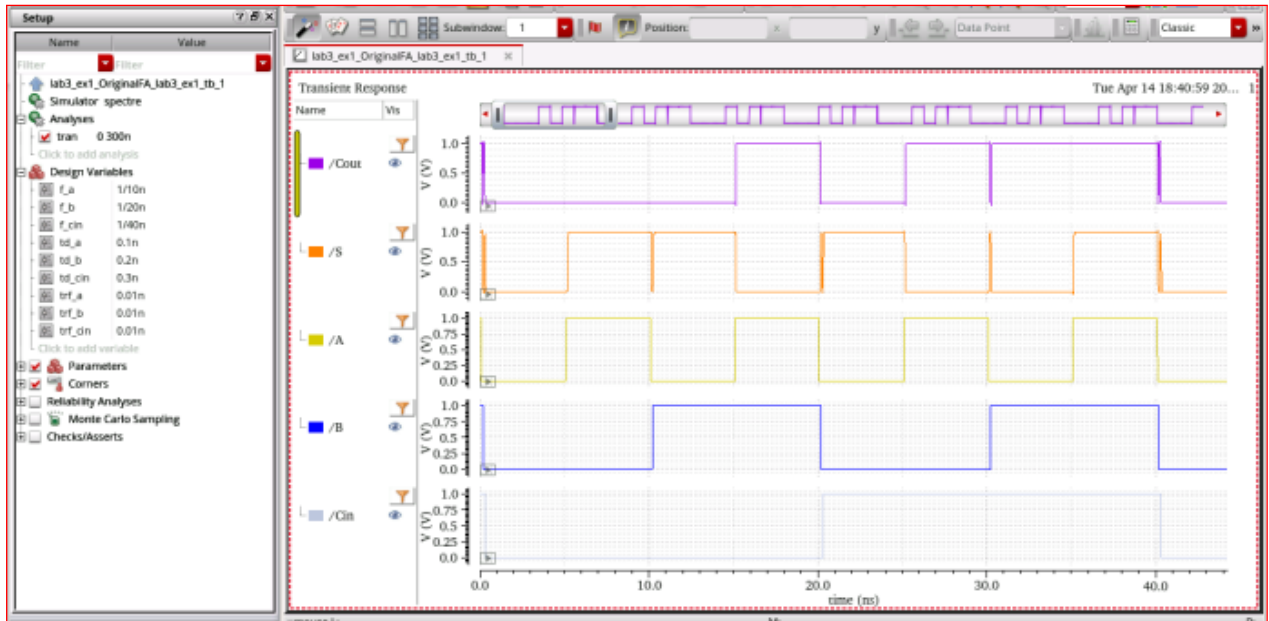


Hình 1: Schematic của cổng Full Adder theo mô hình ghép cổng logic (Original FA)

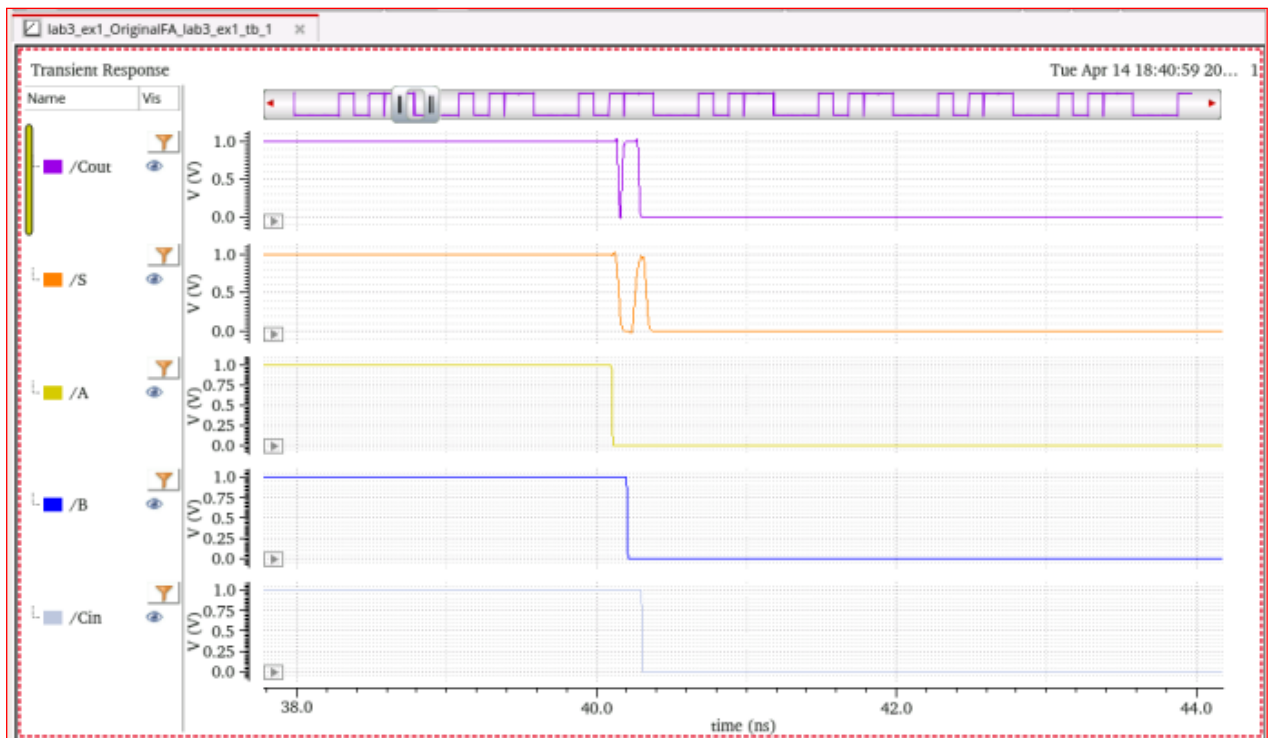
Các cổng logic trên sơ đồ mạch trên được sử dụng lại từ LAB2 với một chút điều chỉnh về symbol để nhìn thuận mắt hơn.



Hình 2: Mạch testbench



Hình 3: Dạng sóng transient



Hình 4: Trạng thái giả tại thời điểm chuyển trạng thái

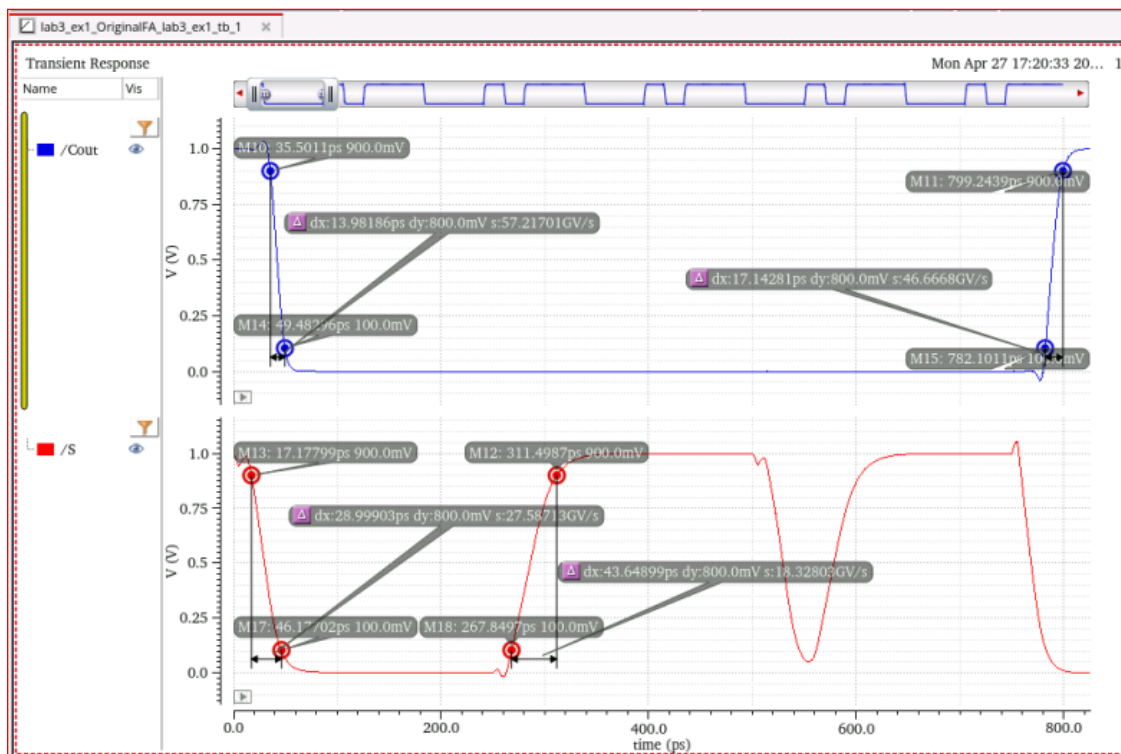
Đối chiếu với bảng chân trị, ta thấy bộ full adder trên đã hoạt động đúng. Tại thời điểm chuyển mạch, các giá trị logic ngõ vào chưa được xác lập (vì C_{in} đến trễ hơn B và B trễ hơn A) nên bộ FA tạo nhiễu (trạng thái giả) tại ngõ ra, thực tế tại các trạng thái giả trên mạch vẫn hoạt động đúng nên ta loại trừ trường hợp mạch sai logic. Ví dụ như tại thời điểm A = 0 đã đến nhưng B và C vẫn giữ tại 1, ngõ ra có giá trị S = 0 Cout = 1 đúng với bảng chân trị chuẩn.

A	B	C_{in}	S	C_{out}	Carry Status
0	0	0	0	0	Delete
0	0	1	1	0	Delete
0	1	0	1	0	Propagate
0	1	1	0	1	Propagate
1	0	0	1	0	Propagate
1	0	1	0	1	Propagate
1	1	0	0	1	Generate
1	1	1	1	1	Generate

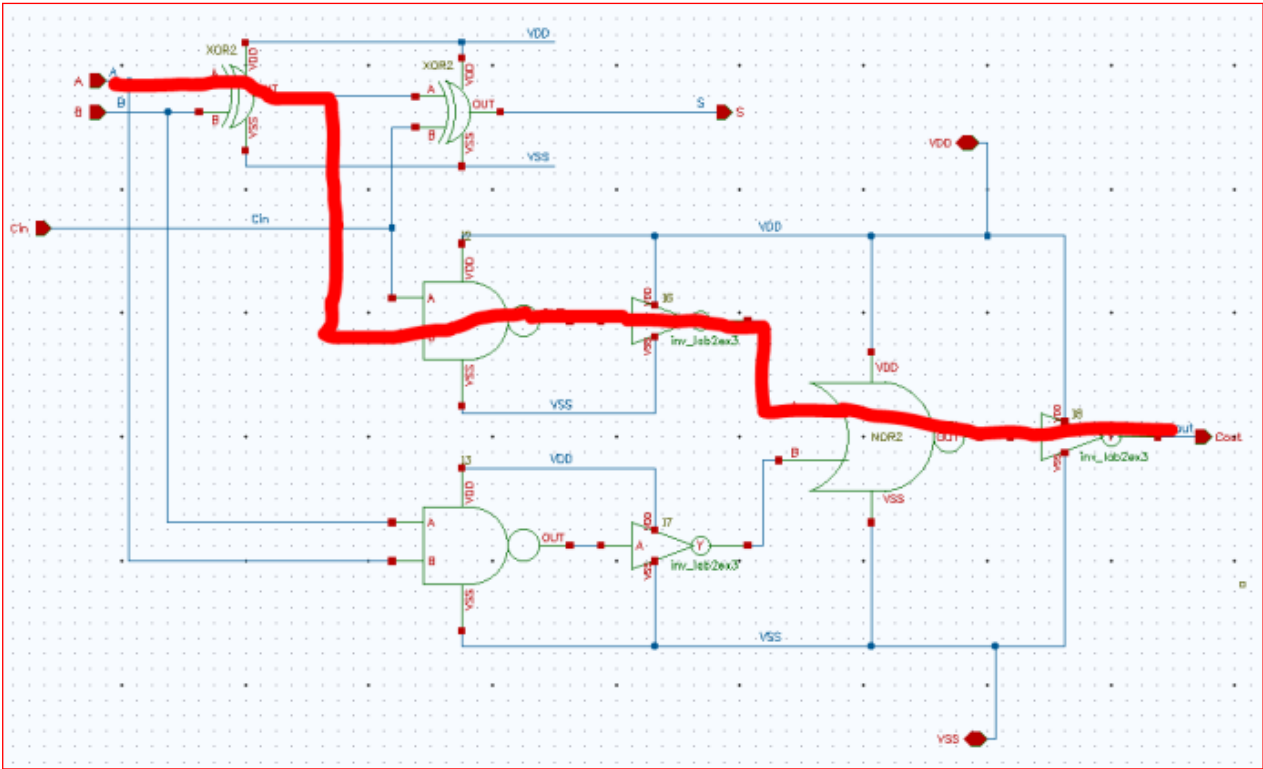
Hình 5: Bảng chân trị của cổng 1-bit Full Adder

Name	Details	Value
t_rise_s	riseTime(VT("/S") 0 nil 1 nil 10 90 nil "time")	43.65p
t_fall_s	fallTime(VT("/S") 0 nil 1 nil 90 10 nil "time")	29p
t_rise_cout	riseTime(VT("/Cout") 0 nil 1 nil 10 90 nil "time")	17.14p
t_fall_cout	fallTime(VT("/Cout") 0 nil 1 nil 90 10 nil "time")	13.98p
P_consumption	average((VT("/VDD") * IT("/I0/VDD")))	13.72u

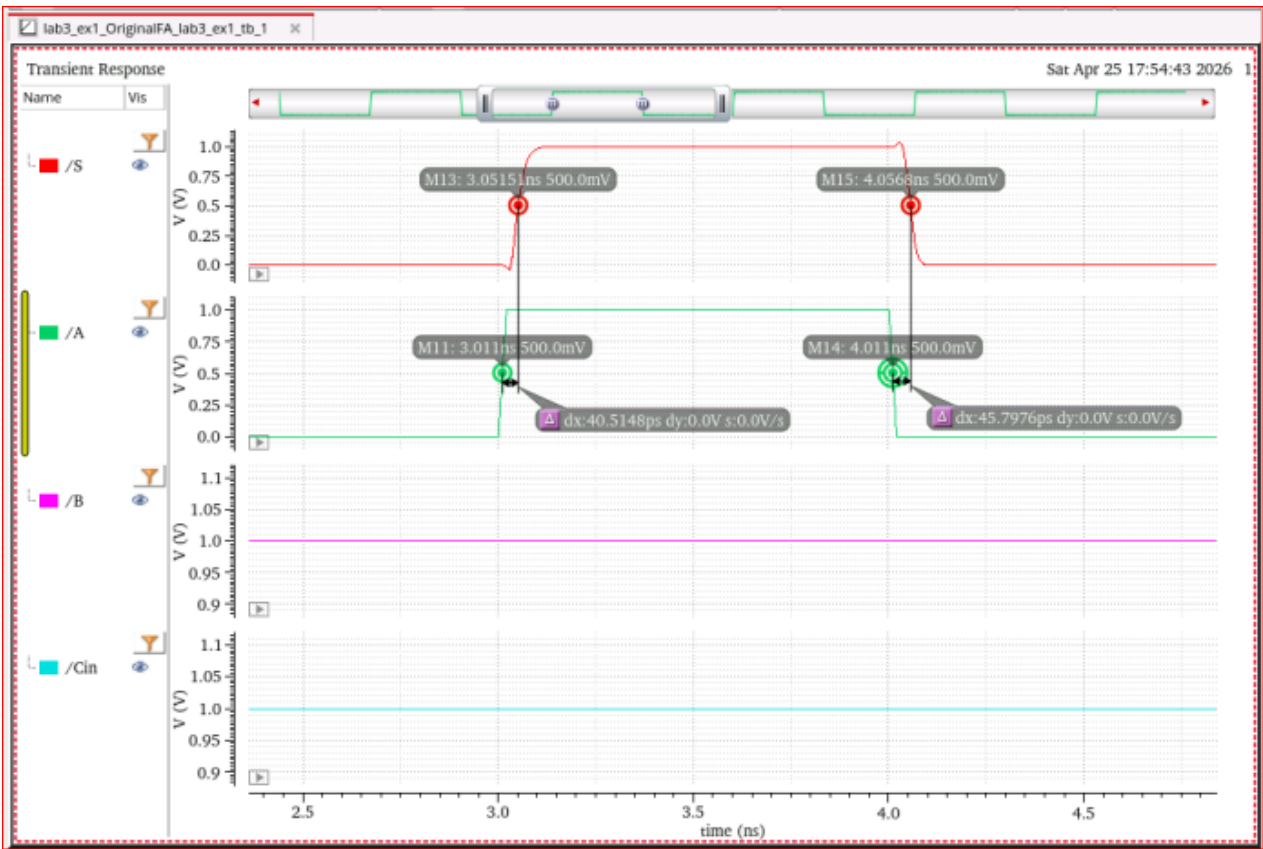
Hình 6: Hàm đo được cài đặt để đo các thông số timing và công suất tiêu thụ



Hình 7: Xác minh các hàm đo



Hình 8: Critical path của cổng basic 1-bit Full Adder



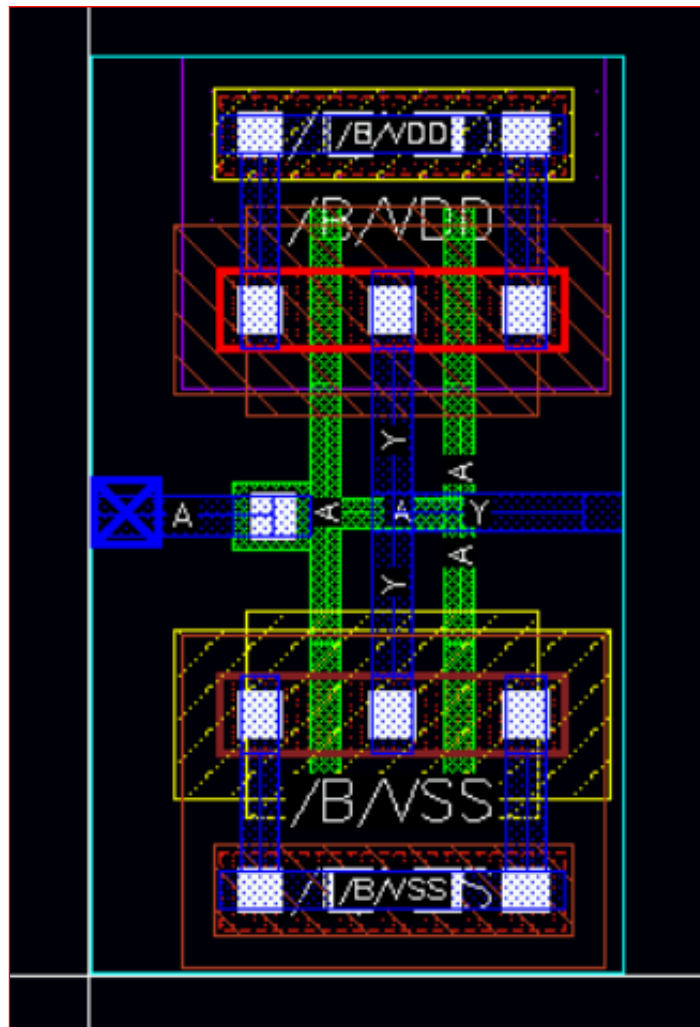
Hình 9: Do trễ lan truyền từ A đến ngõ ra S

Vậy: $t_{pd} = (t_{pdr} + t_{pdf})/2 = (40.5148ps + 45.7976ps)/2 = 43.1562ps$. Tổng hợp kết quả đo:

Parameters	Results
T_rise_s	43.65ps
T_fall_s	29ps
T_rise_cout	17.14ps
T_fall_cout	13.98ps
T_pd	43.1562ps
P_consumption	13.72uW

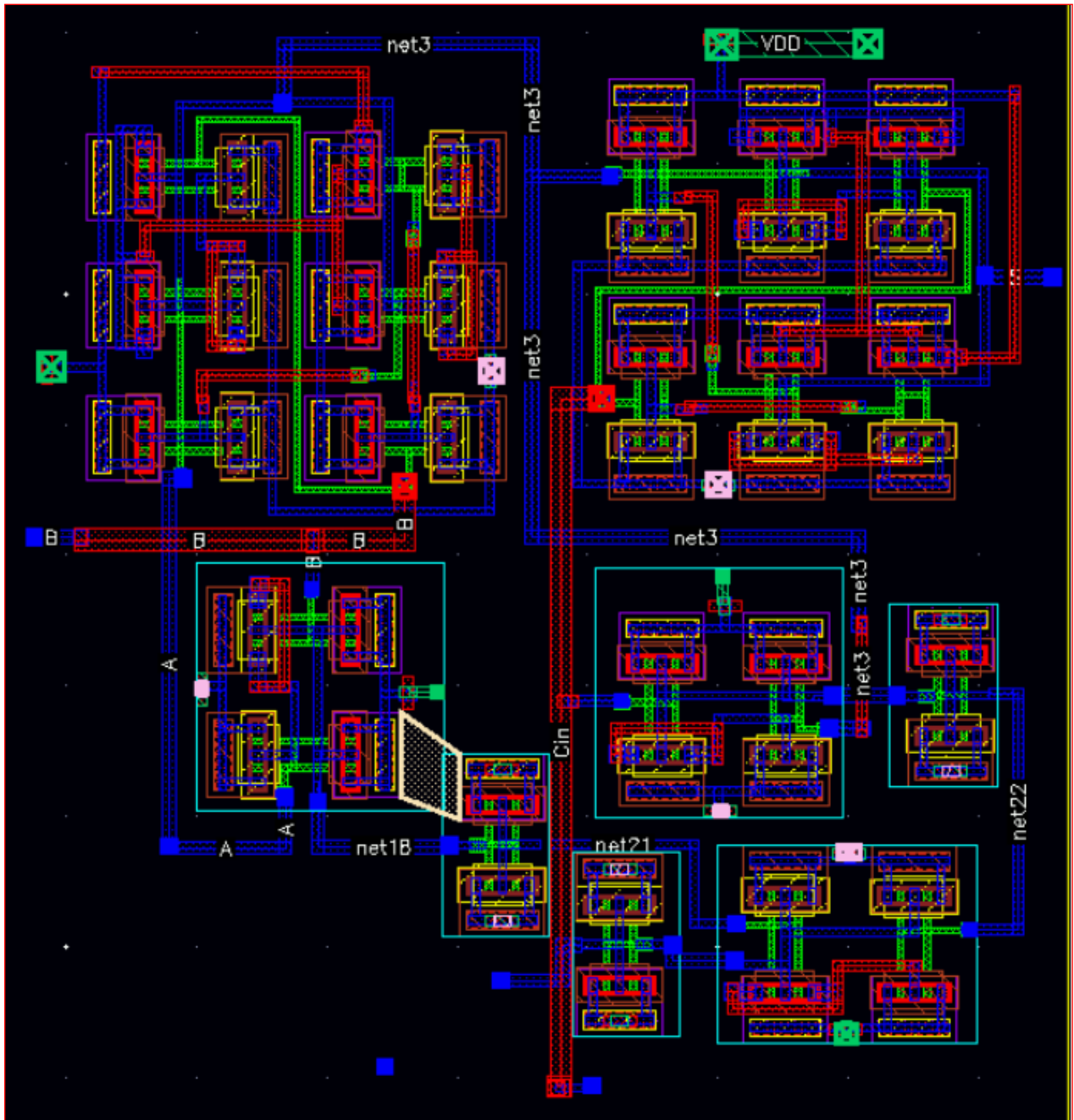
Từ đó ta tìm được max speed như sau: $f_{max} \approx \frac{1}{t_{pd,critical}} = \frac{1}{43.1526p} = 23.17GHz$

LAYOUT: Vẽ thêm layout cho cổng inverter từ lab2:



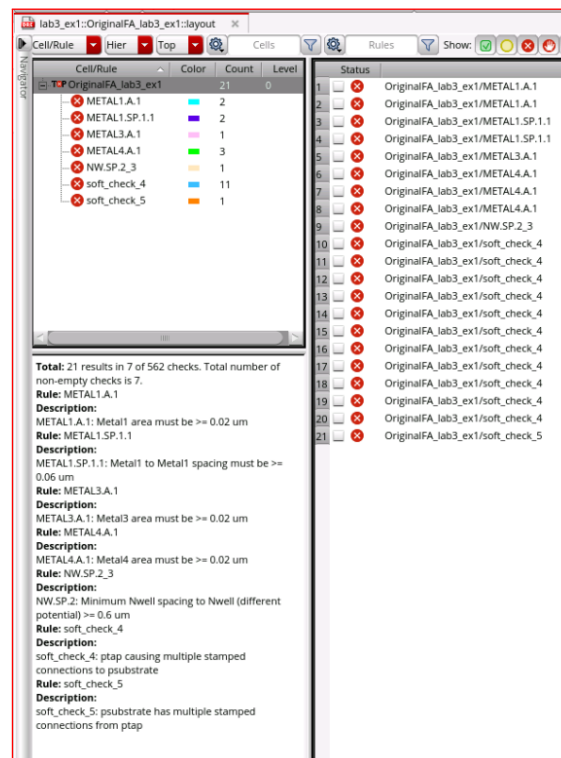
Hình 10: Layout của cổng inverter được lấy từ lab2 ex3

Ta tiến hành sắp xếp vị trí và đi dây hợp lý cho các phần tử logic trước và check DRC trước khi cấp nguồn cho thiết kế:



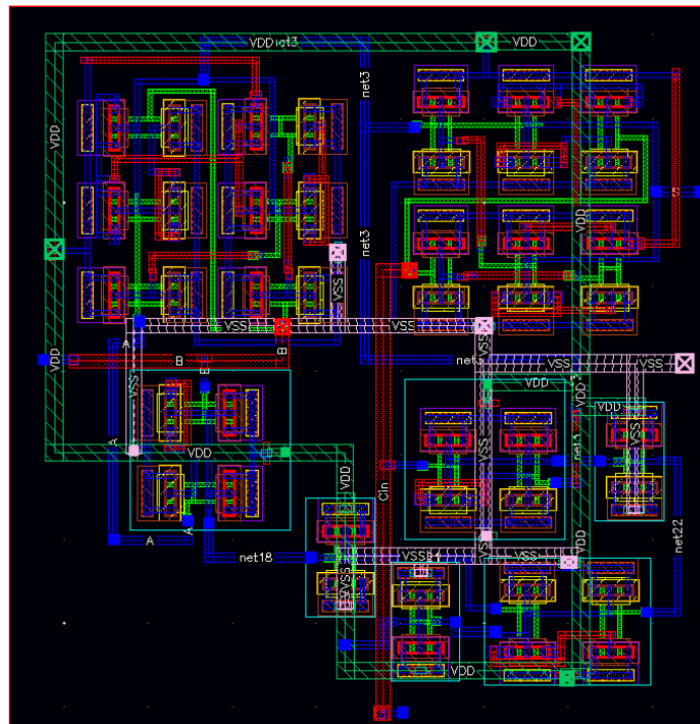
Hình 11: Layout sơ bộ của cổng Original 1-bit FA trước khi đi các đường VDD và VSS

DRC reports:



Hình 12: kết quả khi chạy DRC lần đầu

Ta tiến hành sửa tất cả các lỗi trên, có thể thấy thiết kế của ta không gặp lỗi nghiêm trọng, các lỗi chỉ chủ yếu đến từ vấn đề spacing giữa các đường dây và NWell của FET.



Hình 13: Layout sau khi hoàn thành check DRC cho các phần tử logic và tiến hành cấp nguồn cho thiết kế

Hoàn thành bước kiểm tra DRC:

```

Total CPU Time      : 2(s)
Total Real Time     : 34(s)
Peak Memory Used    : 25(M)
Total Original Geometry : 642(2177)
Total DRC RuleChecks : 562
Total DRC Results   : 0 (0)
Summary can be found in file OriginalFA_lab3_ex1.sum
ASCII report database is /home/yellow/ee3117_8/DIC_lab/lab3/lab3_ex1/DRC_28t/OriginalFA_lab3_ex1.drc_errors.ascii
Checking in all SoftShare licenses.

Design Rule Check Finished Normally. Tue Apr 21 00:22:11 2026

```

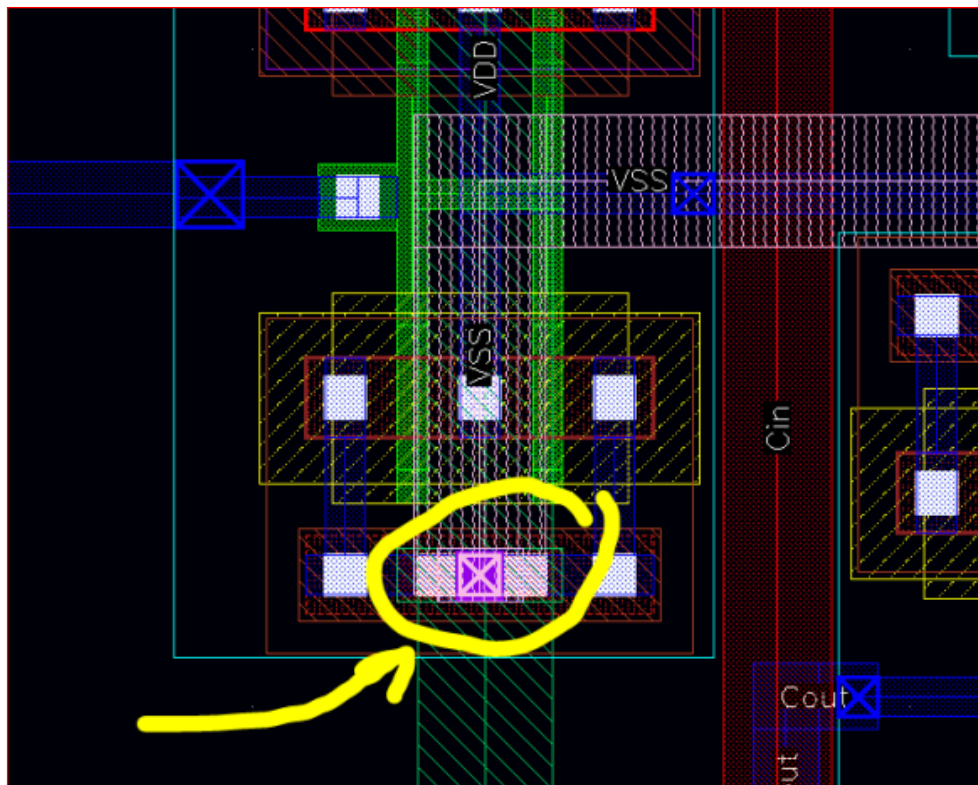
Hình 14: kết quả khi hoàn thành DRC

LVS reports: khi chạy LVS layout của ta gặp một số lỗi nghiêm trọng:

Lỗi 1: short giữa VDD và VSS:

OriginalFA_lab3_ex1 : SHORTS AND OPENS	
Layout Net: 4	Schematic Pin: VDD
	(sao 1)
SHORT	Schematic Pin: VSS

Nguyên nhân:



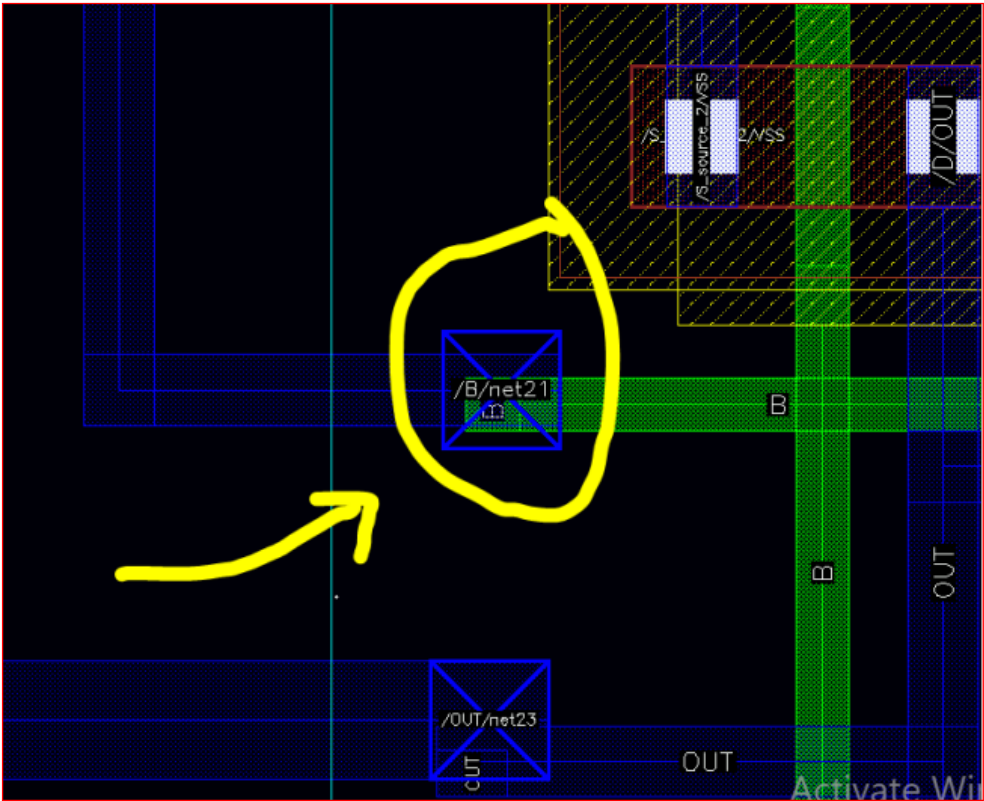
Hình 15: Via từ Metal1 đến metal4 gây short với metal3 của VDD

⇒ Cách sửa: đi lại VDD tránh các Via của VSS.

Lỗi 2: hở mạch ở ngõ vào B cổng NOR:

OriginalFA_lab3_ex1 : SHORTS AND OPENS	
Layout Net: X5/7	Schematic Net: net21
Layout Net: 12	(sao 1) OPEN

Nguyên nhân:



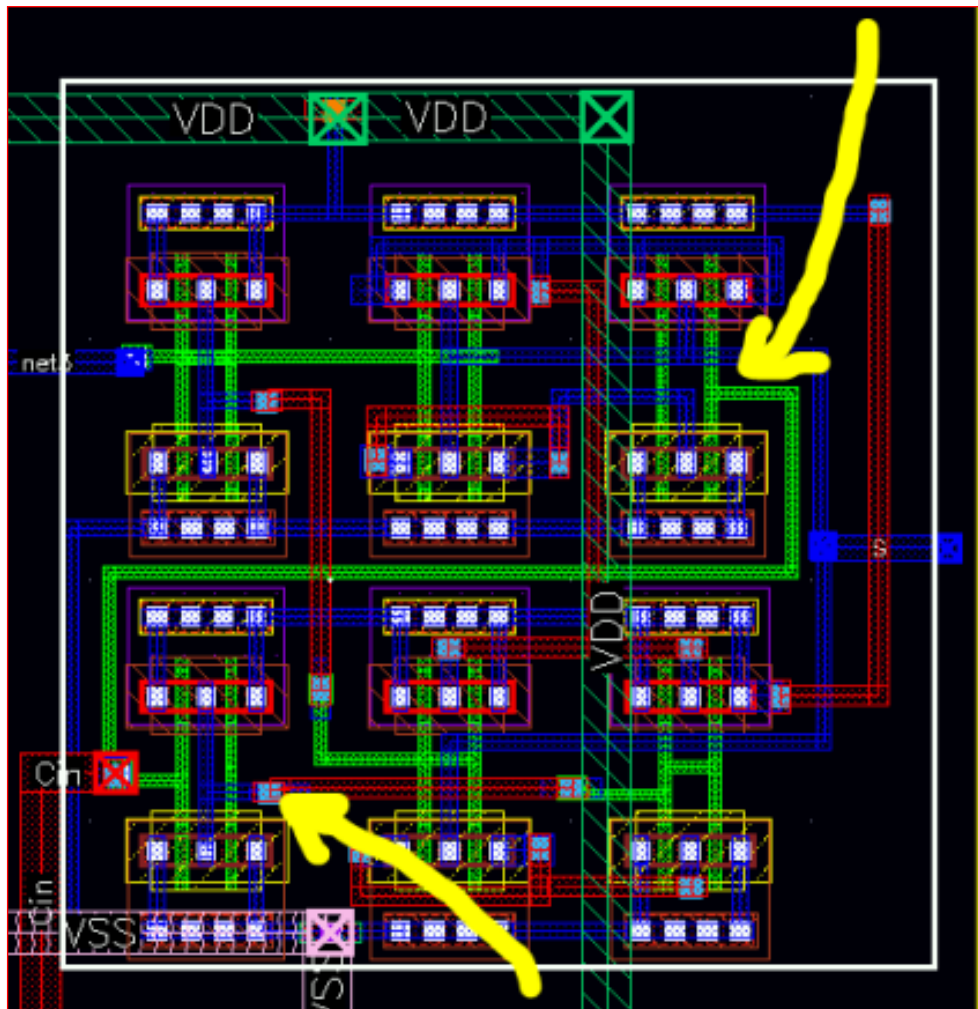
Hình 16: Thiếu Via nối poly với metal1

⇒ Cách sửa: Thêm via.

Lỗi 3: chưa short các cực D của 2 fingers FET lại với nhau trong cổng XOR2

OriginalFA_lab3_ex1 : MISMATCHED INSTANCE PARAMETERS		
Layout Instance	Schematic Instance	Parameter Error
X2/X27/M0 @(9.360,8.815) MN(g45n1lvt) W: 0.12 u	XI1/MNM0 MN(G45N1LVT) w: 0.24 u	50% (mip 1)
X2/X21/M0 @(9.360,9.455) MP(g45p1lvt) W: 0.12 u	XI1/MPM5 MP(G45P1LVT) w: 0.24 u	50% (mip 2)
X1/X27/M0 @(7.980,8.775) MN(g45n1lvt) W: 0.12 u	XI0/MNM0 MN(G45N1LVT) w: 0.24 u	50% (mip 3)
X1/X21/M0 @(7.340,8.775) MP(g45p1lvt) W: 0.12 u	XI0/MPM5 MP(G45P1LVT) w: 0.24 u	50% (mip 4)

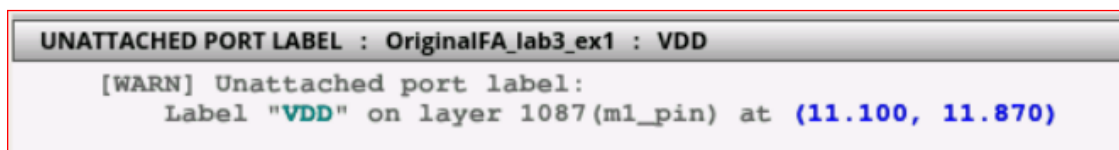
Nguyên nhân:



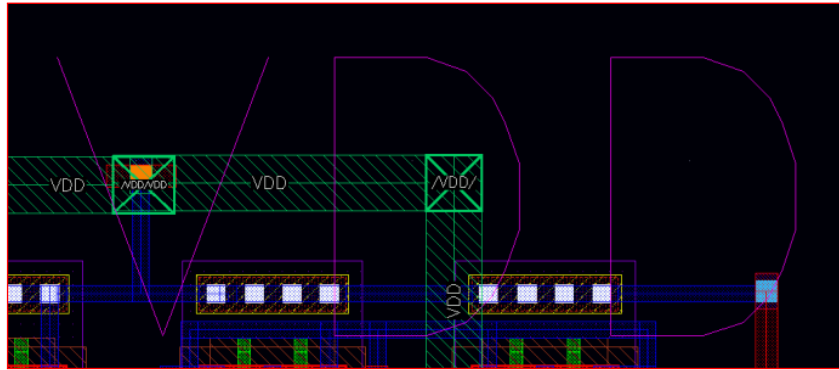
Hình 17: Các cực G của 2 body chưa được nối với nhau

⇒ Cách sửa: nối 2 dây poly lại với nhau.

Lỗi 4: các label nằm ở layer khác với pin hiện tại:

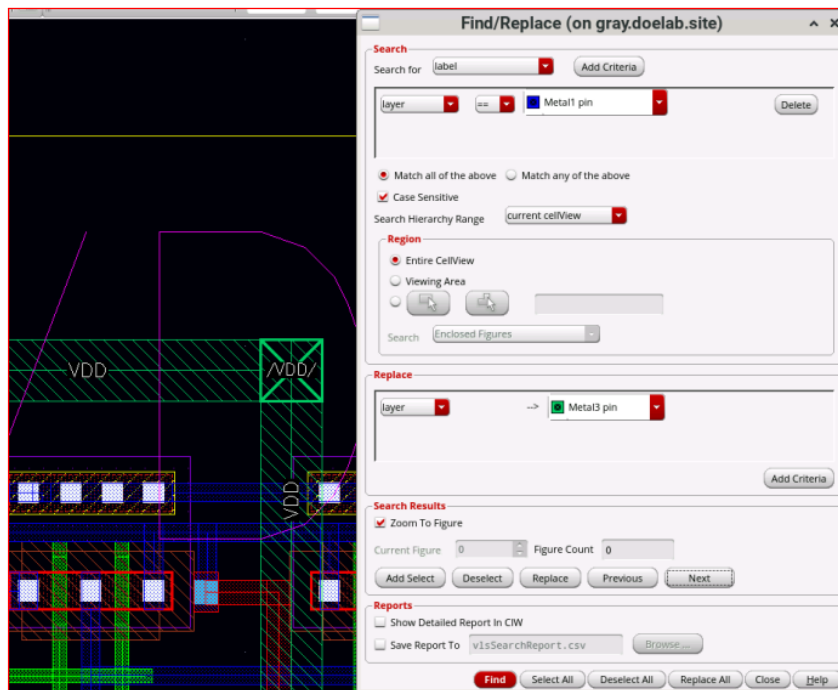


Nguyên nhân: Ban đầu label được đặt theo pin tại metal1, nhưng trong quá trình vẽ pin đã được đẩy lên metal3, vậy ta cần cài đặt lại để label cũng lên metal3.



Hình 18: Lỗi label nằm ở layer khác

⇒ Cách sửa: Dùng công cụ find/replace để đặt lại thuộc tính cho các label.

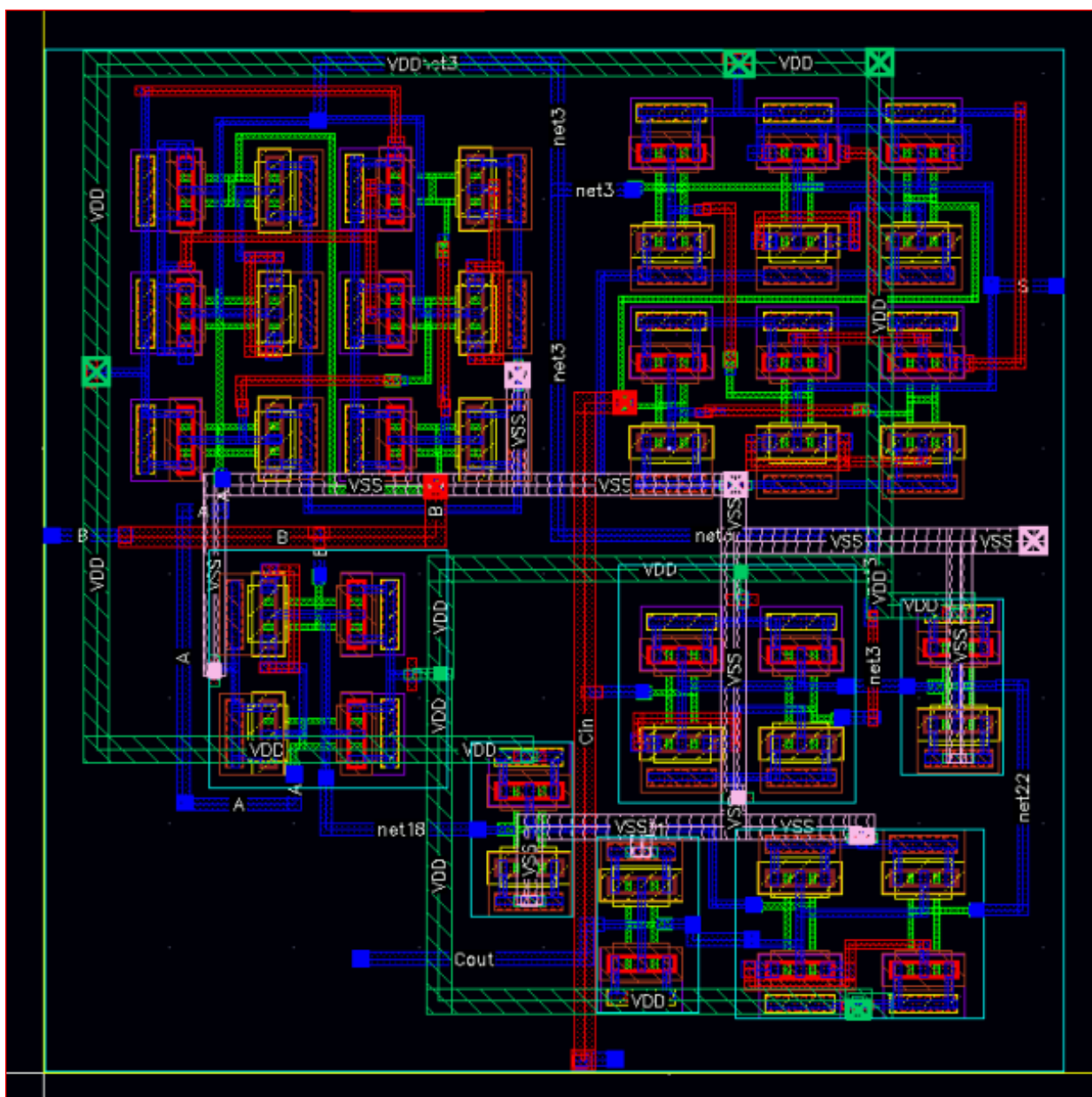


```
#####
#
# Run Result           : MATCH
#
# Run Summary          : [WARN] Extraction Warnings (refer to Extraction Report File)
#                      : [INFO] ERC Results: Empty
#
# ERC Summary File     : OriginalFA_lab3_ex1.sum
# Extraction Report File : OriginalFA_lab3_ex1.rep
# Comparison Report File : OriginalFA_lab3_ex1.rep.cls
#
#####
Checking in all SoftShare licenses.

PVS Comparison Finished. Tue Apr 21 11:23:52 2026
```

Hình 19: kết quả sau khi hoàn thành sửa lỗi LVS

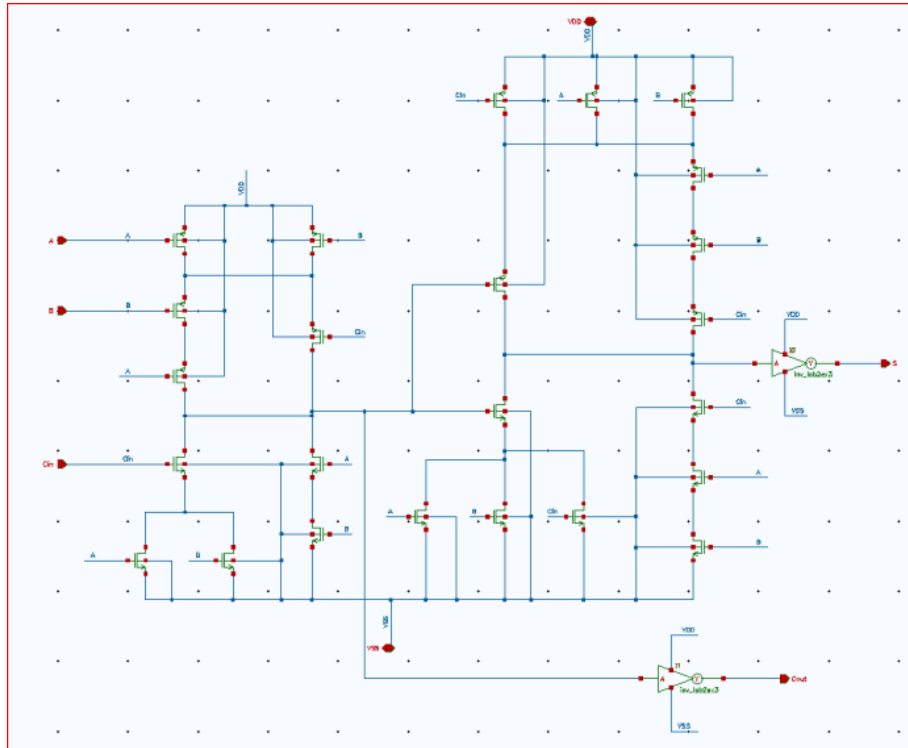
Kết quả layout cuối cùng:



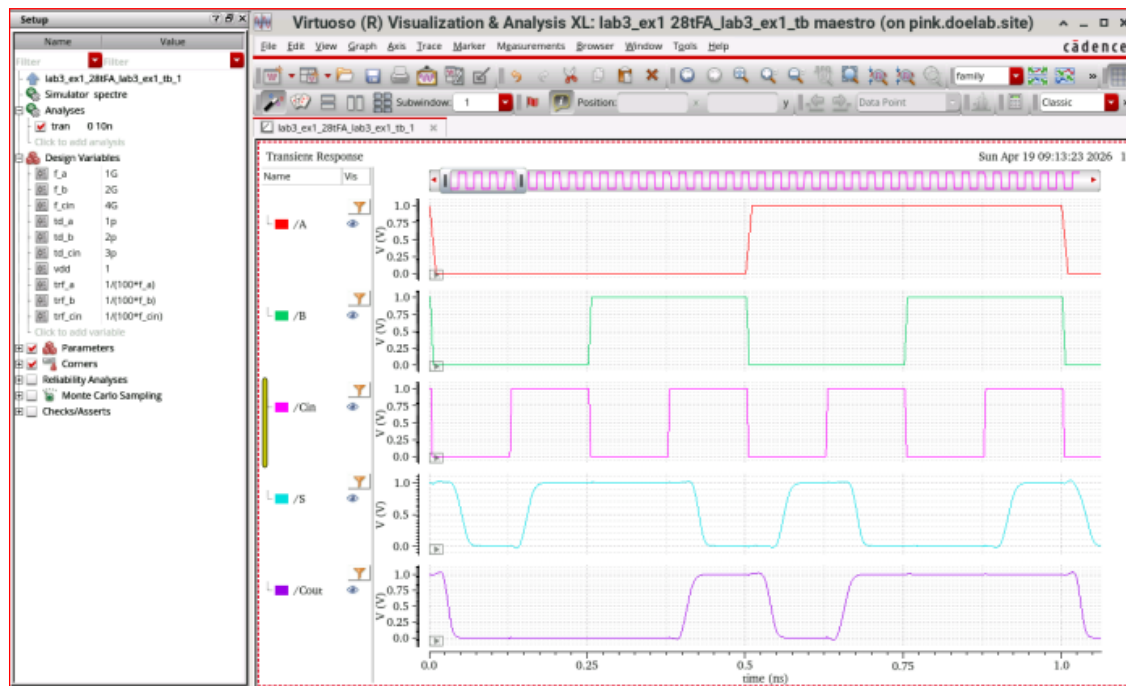
Hình 20: Kết quả layout hoàn thiện của cổng Original Full Adder

1.2 28T Full Adder

1.2.1 Xác minh thiết kế và đo đặc timing, công suất tiêu thụ



Hình 21: Schematic của cổng 28T Full Adder



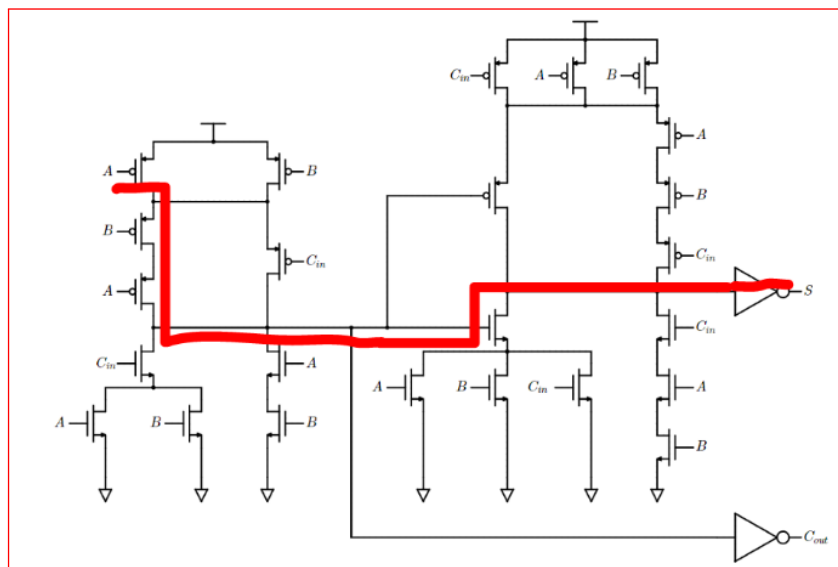
Hình 22: Dạng sóng xác minh thiết kế tại tất cả các trường hợp

Đối chiếu với bảng chân trị ở hình 5, ta thấy cổng 28T FA trên đã hoạt động đúng.

Đo các thông số timing và tính max speed: Các hàm đo được sử dụng cho mạch này được áp dụng lại từ các hàm đã dùng cho mạch OriginalFA trước đó vì bản chất hoàn toàn tương đồng của chúng.

Name	Details	Value
t_rise_s	riseTime(VT("/S") 0 nil 1 nil 10 90 nil "time")	20.18p
t_fall_s	fallTime(VT("/S") 0 nil 1 nil 90 10 nil "time")	20.95p
t_rise_cout	riseTime(VT("/Cout") 0 nil 1 nil 10 90 nil "time")	20p
t_fall_cout	fallTime(VT("/Cout") 0 nil 1 nil 90 10 nil "time")	13.68p
P_consumption	average((VT("/VDD") * IT("/IO/VDD")))	15.51u

Hình 23: Hàm đo được sử dụng để tìm các thông số timing cùng công suất tiêu thụ



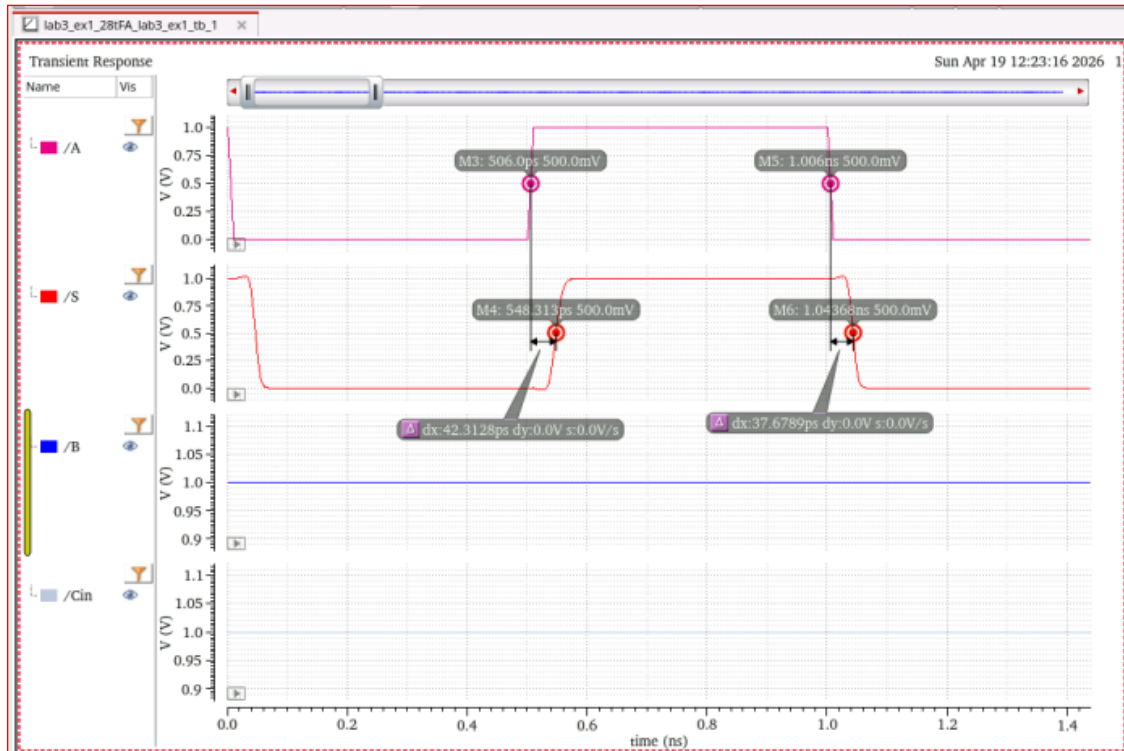
Hình 24: Critical path của cổng basic 1-bit Full Adder

Để đo được t_{pd} từ A đến S ta cần cài đặt testbench dựa trên tổ hợp mà tại đó S thay đổi theo A.

A	B	C_{in}	S	C_{out}	Carry Status
0	0	0	0	0	Delete
0	0	1	1	0	Delete
0	1	0	1	0	Propagate
0	1	1	0	1	Propagate
1	0	0	1	0	Propagate
1	0	1	0	1	Propagate
1	1	0	0	1	Generate
1	1	1	1	1	Generate

Hình 25: các tổ hợp được lựa chọn để cài đặt testbench đo t_{pd}

Theo bảng chân trị ta quyết định set B, C_{in} luôn bằng 1 và A dao động để đo trễ lan truyền từ A đến S.

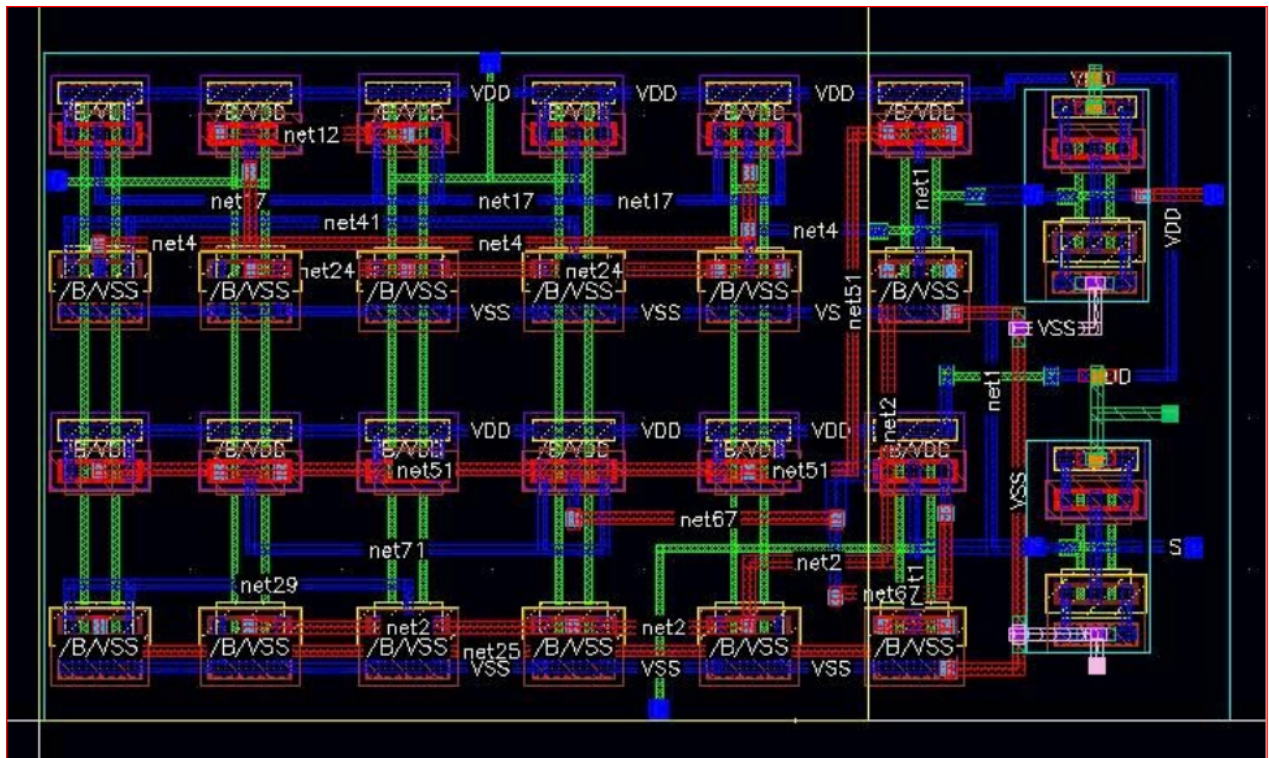


Hình 26: kết quả đo t_{pd} của 28T fulladder

Vậy: $t_{pd} = (t_{pdr} + t_{pdf})/2 = (42.3128ps + 37.6789ps)/2 = 39.99585ps$. Tổng hợp các kết quả đã đo được:

Parameters	Results
T_{rise_s}	20.18ps
T_{fall_s}	20.95ps
T_{rise_cout}	20ps
T_{fall_cout}	13.68ps
T_{pd}	39.99585ps
$P_{consumption}$	15.15uW

$$\text{Từ đó ta có max speed: } f_{max} \approx \frac{1}{t_{pd,critical}} = \frac{1}{39.99585p} = 25.0026GHz$$

Layout:

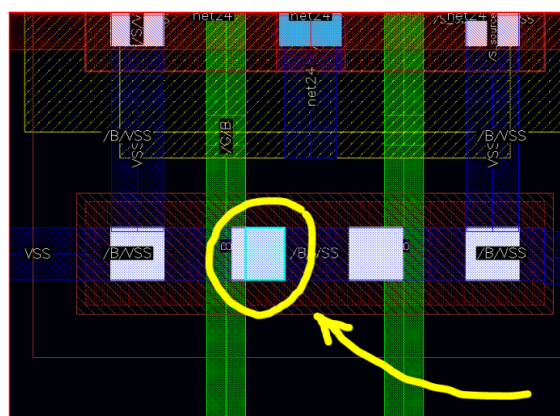
Hình 27: Layout trước khi thực hiện DRC và LVS

DRC Results:

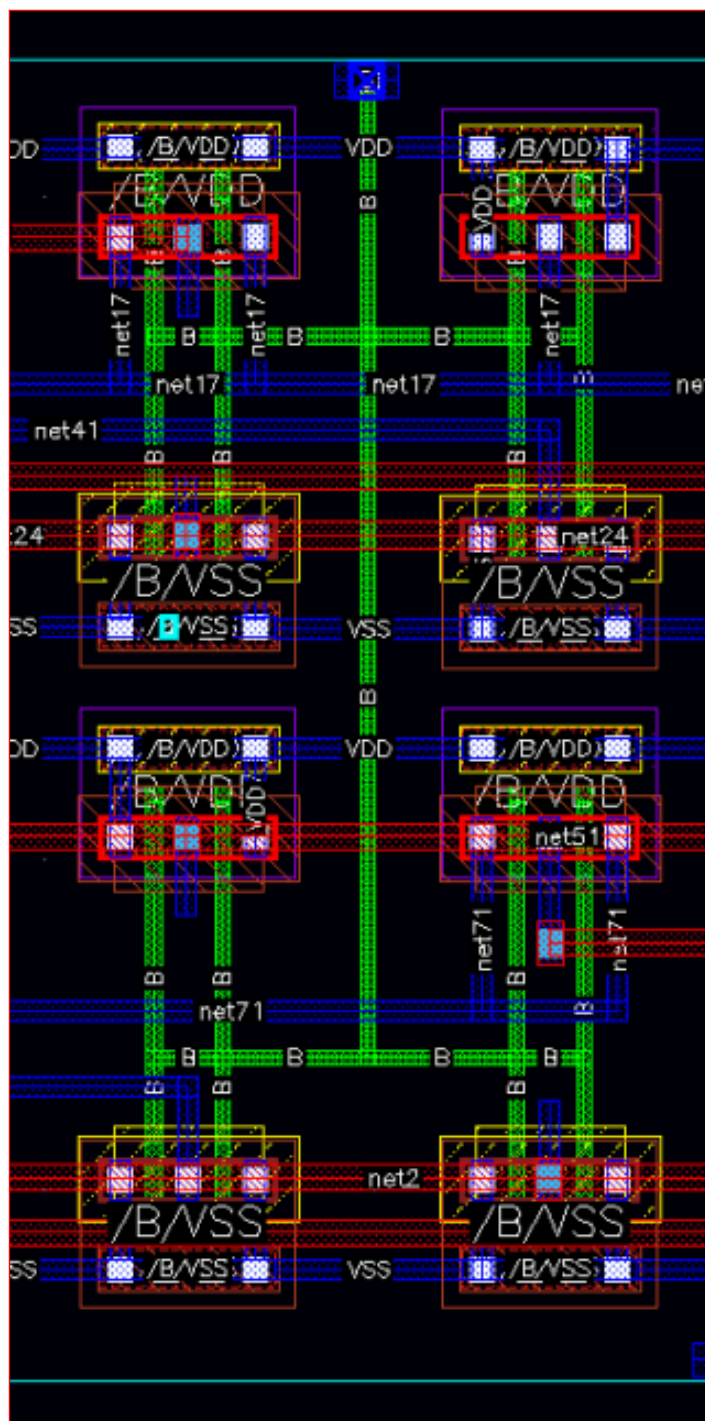
Lỗi 1: Poly trùng lên cực B

Total: 46 results in 11 of 562 checks. Total number of non-empty checks is 11.
Rule: CONT.E.2
Description:
 CONT.E.2: Minimum Poly to Contact enclosure ≥ 0.02 um

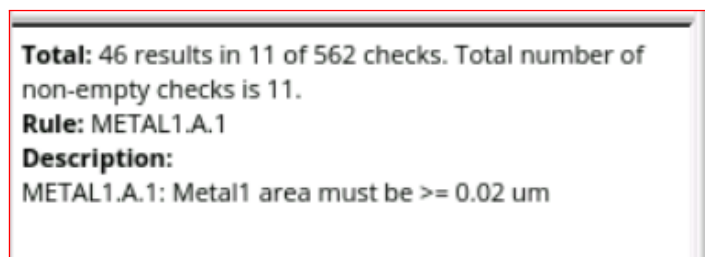
Nguyên nhân: các terminal của cực B nổi từ diffusion lên metall có đi qua tầng poly, nếu nổi poly vào sẽ gây ngắn mạch.



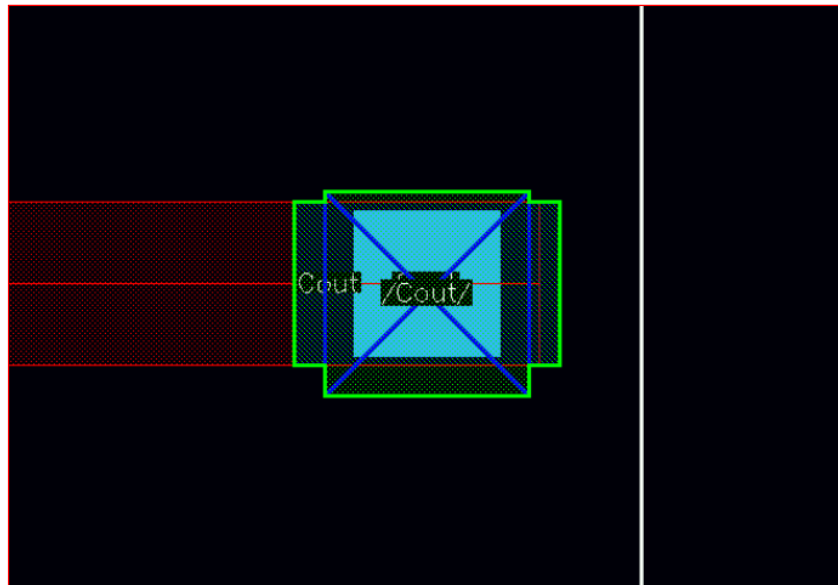
⇒ Cách sửa: đi poly lại để tránh các cực B:



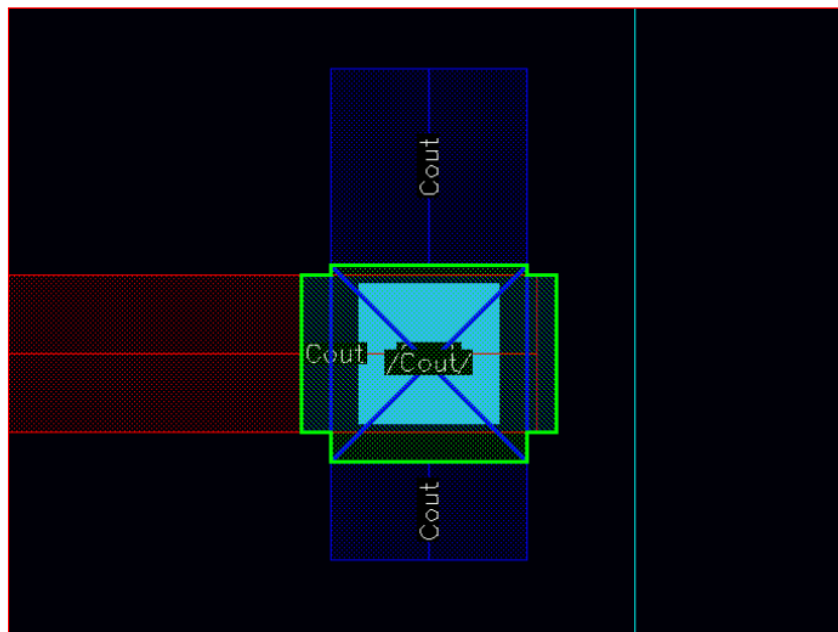
Lỗi 2: Area quá nhỏ



Nguyên nhân: pin Cout tại metal1 không đáp ứng area tối thiểu cho quá trình fabchip



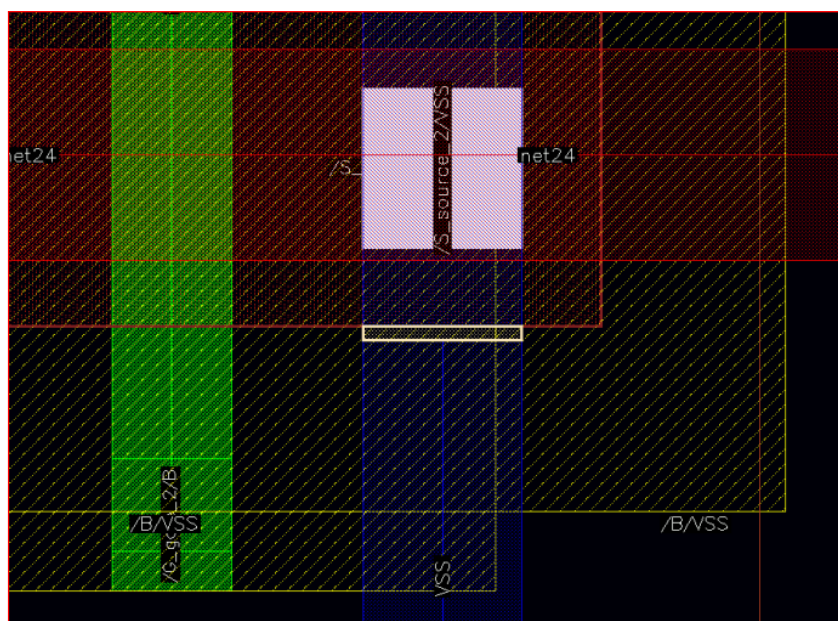
⇒ Cách sửa: tăng area



Lỗi 3: Hở mạch

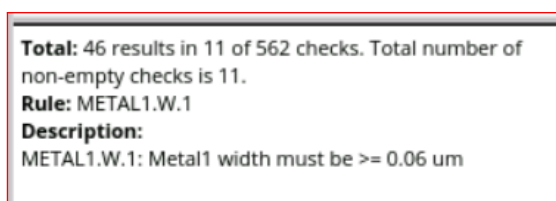
Total: 46 results in 11 of 562 checks. Total number of non-empty checks is 11.
Rule: METAL1.SP.1.1
Description:
METAL1.SP.1.1: Metal1 to Metal1 spacing must be $\geq 0.06 \mu\text{m}$

Nguyên nhân: đường dây metal1 bị hở

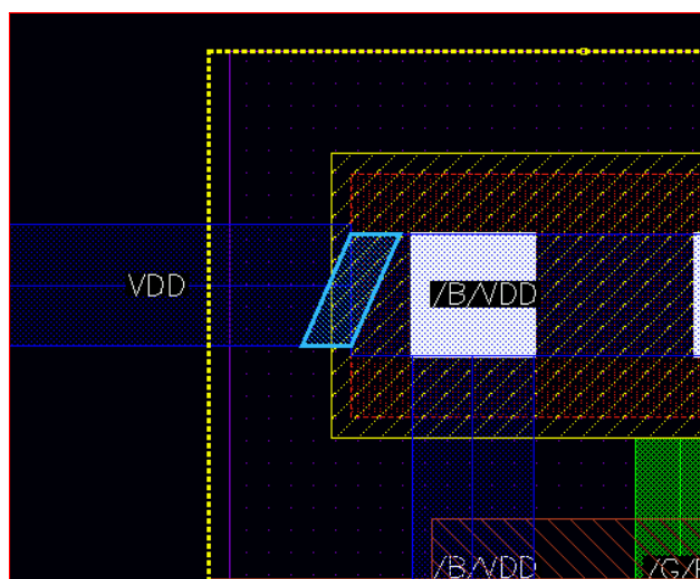


⇒ Cách sửa: nối kín đường dây trên.

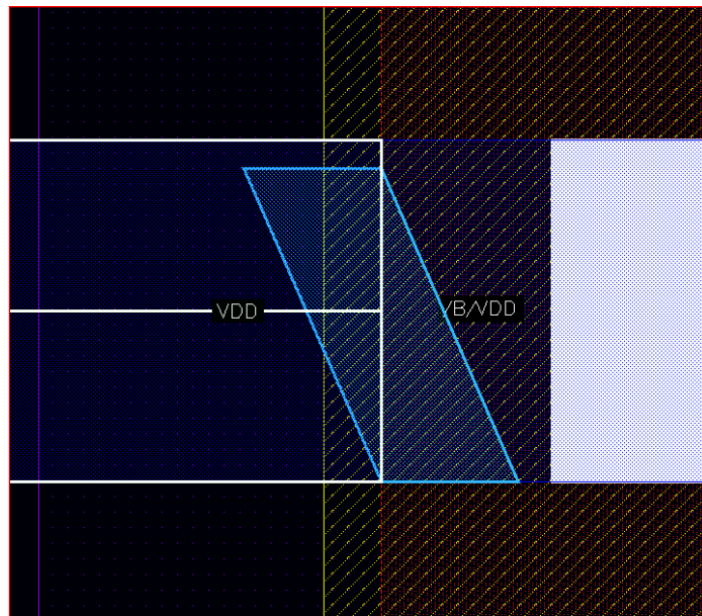
Lỗi 4:



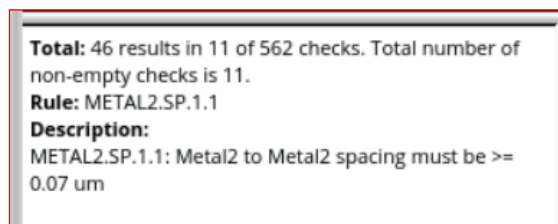
Nguyên nhân:



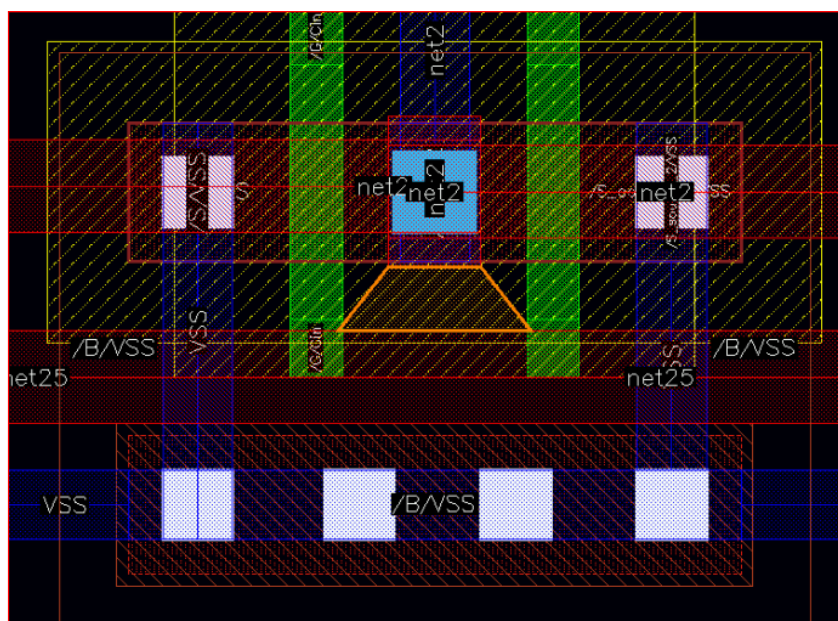
⇒ Cách sửa: nắn thẳng đường dây trên.



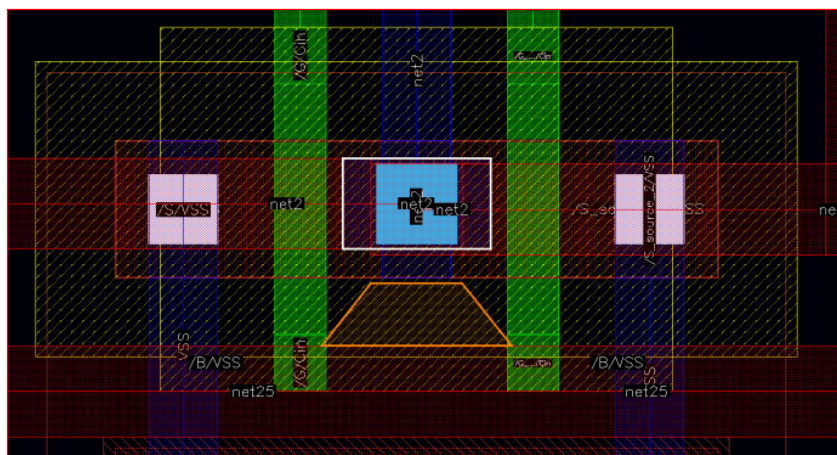
Lỗi 5: khoảng cách quá gần giữa 2 đường metal2



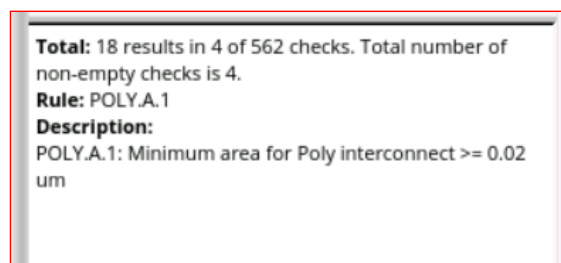
Nguyên nhân: via từ terminal của cực D mosfet lên metal2 nằm quá gần một đường metal2 khác bên cạnh gây violate



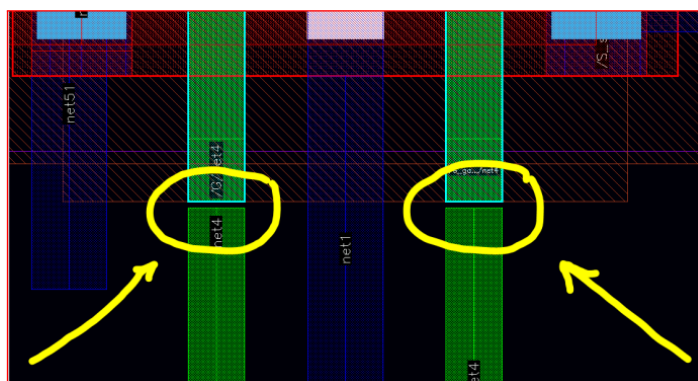
⇒ Cách sửa: xoay ngang via để đảm bảo khoảng cách



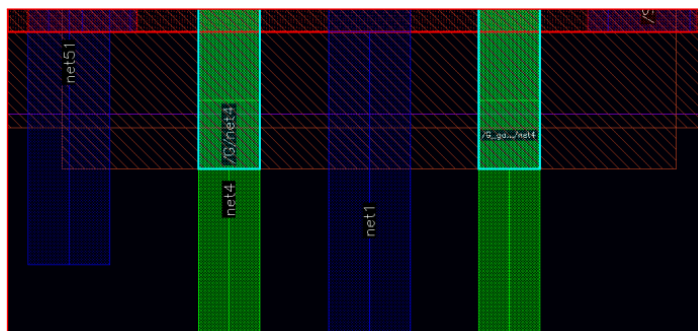
Lỗi 6: hở mạch

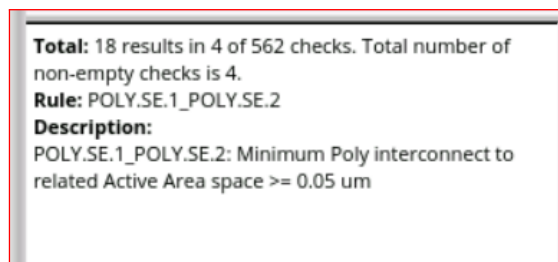


Nguyên nhân:

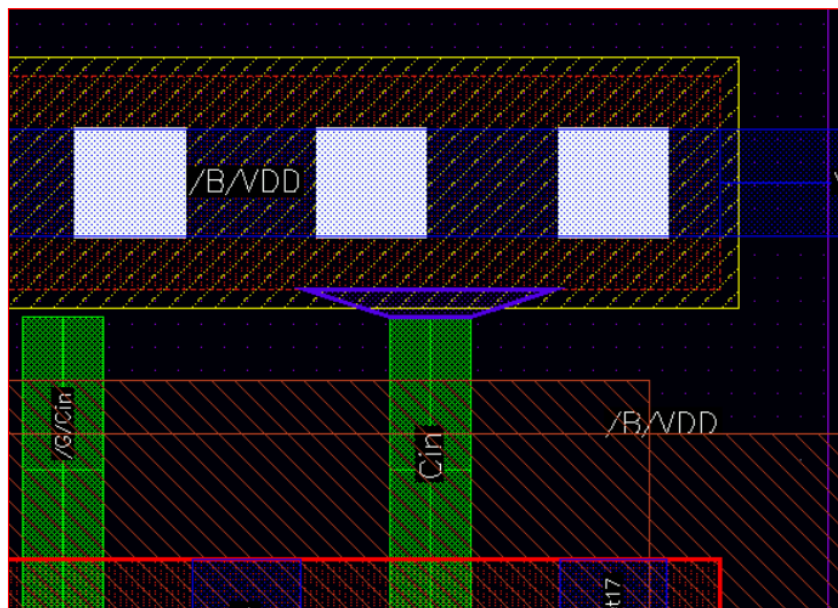


Cách sửa: nối lại khoảng hở

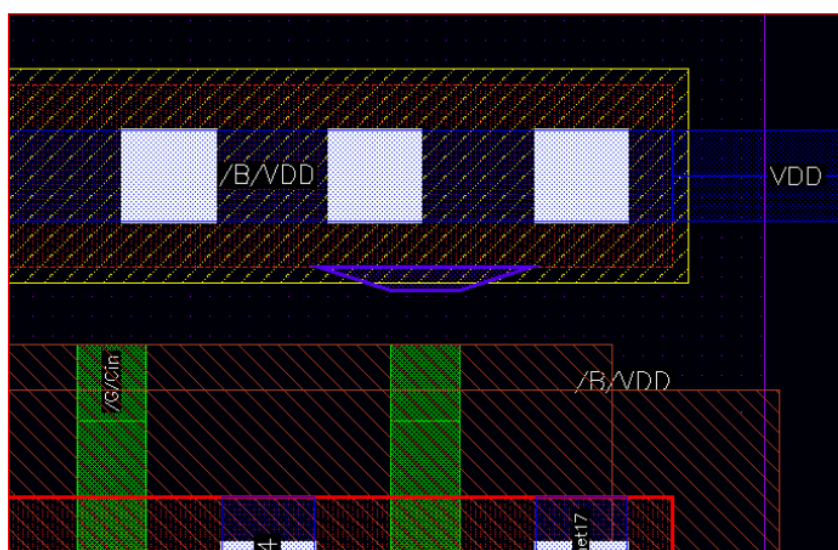


Lỗi 7: poly nằm quá gần active area

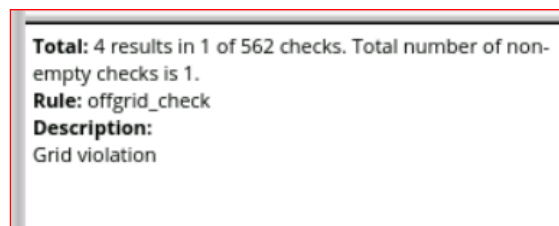
Nguyên nhân: poly được thiết kế để có tương tác về điện trường với các active area qua đó tạo nên MOSFET, nếu đặt chúng quá gần nhau sẽ gây sai lệch thông số mạch



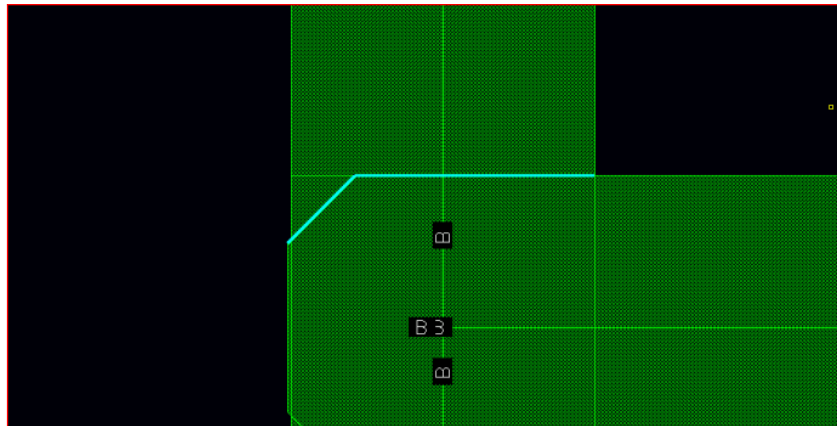
Cách sửa: rút ngắn chân G (poly)



Lỗi 8: xuất hiện góc khác 90 độ



Nguyên nhân:



Cách sửa: đi lại đường dây để không xuất hiện các góc khác 90 độ.

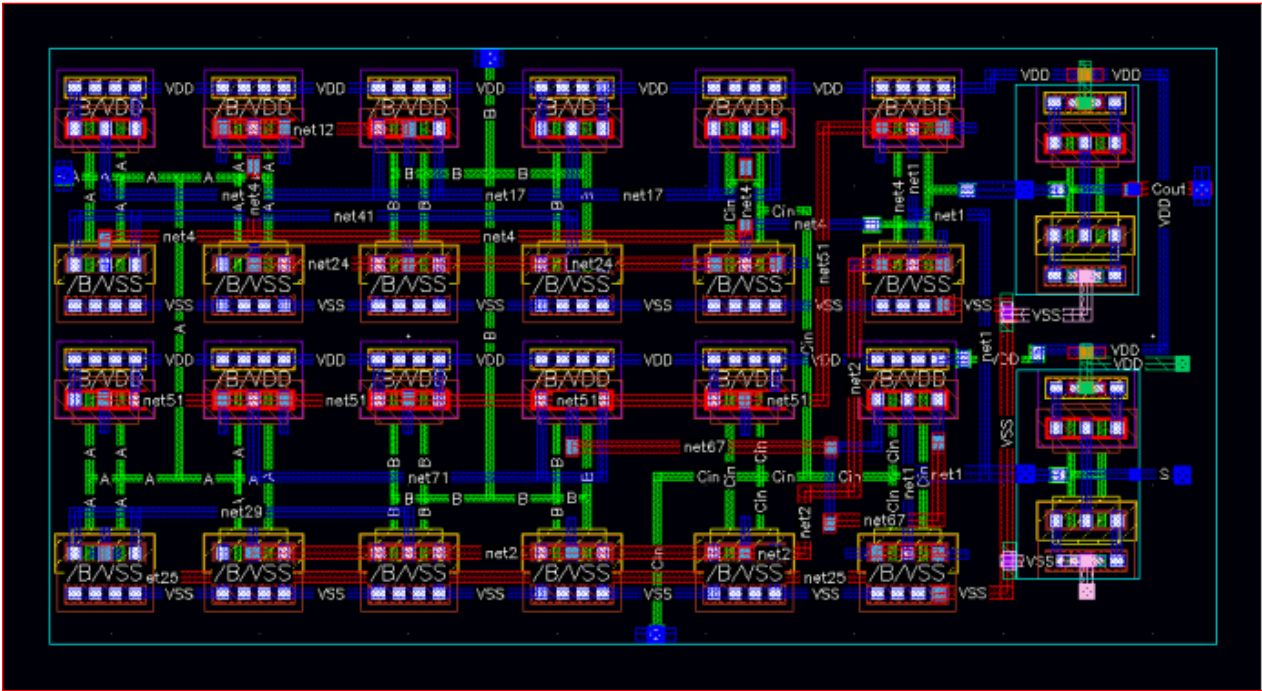


Hoàn thành DRC:

```
Total CPU Time           : 1 (s)
Total Real Time          : 20 (s)
Peak Memory Used         : 25 (M)
Total Original Geometry  : 402 (1384)
Total DRC RuleChecks     : 562
Total DRC Results        : 0 (0)
Summary can be found in file 28tFA_lab3_ex1.sum
ASCII report database is /home/yellow/ee3117_8/DIC_lab/lab3/lab3_ex1/DRC/28tFA_lab3_ex1.drc_errors.ascii
Checking in all SoftShare licenses.

Design Rule Check Finished Normally. Wed Apr 22 15:39:41 2026
```

Hình 28: kết quả sau khi hoàn thành sửa lỗi DRC



Hình 29: Layout sau bước DRC

LVS results:

Lỗi: không tìm thấy pin labels

Files ☒ ERC ☒ Extraction ☒ Comparison

Mismatched

Group by Cell Error Count Status Details

28tFA_lab3_ex1 pin mismatches

UNMATCHED SCHEMATIC PIN LABELS

28tFA_lab3_ex1 : UNMATCHED SCHEMATIC PIN LABELS

Labeled Schematic Pin	Matched Layout Pin	Matched Layout Net
A	** missing pin **	9
B	** missing pin **	10
Cin	** missing pin **	11
Cout	** missing pin **	13
S	** missing pin **	19
VDD	** missing pin **	12
VSS	** missing pin **	8

Nguyên nhân: các pin chưa được đặt label

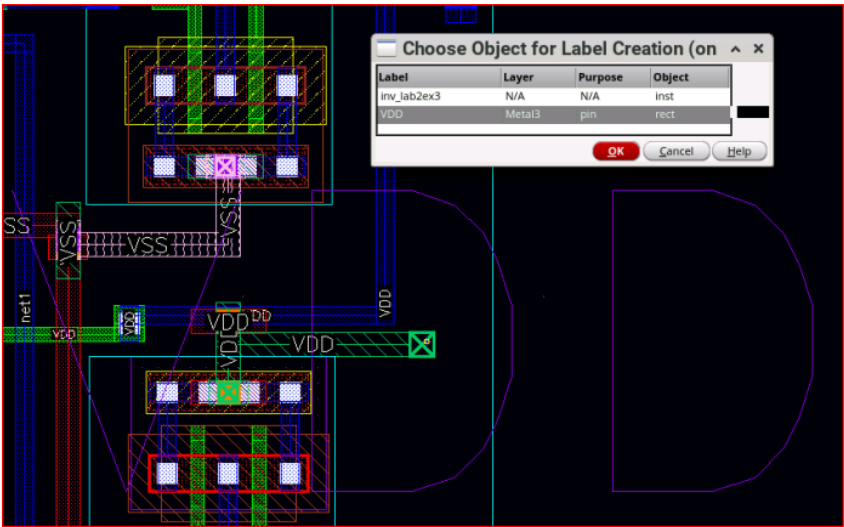
Pin Tool - Pin Browser (on white.doelab.site)

Create Place Tools View Help

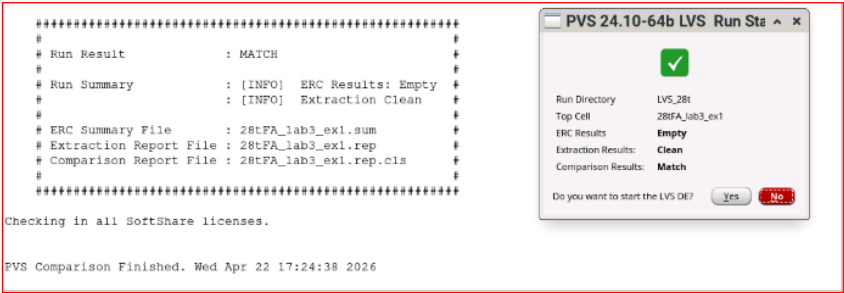
Search...

Pin Name/ Instance Name	Lpp	Width	Height	Placer/Status	Direction	Signal Type	Edge	Preserve Order	Has Label	Schematic
A:AA	Metal1 pin	0.1	0.1	P	input	signal	Any	No	No	Yes
B:BB	Metal1 pin	0.1	0.1	P	input	signal	Any	No	No	Yes
Cin:Cin:Cin	Metal1 pin	0.1	0.1	P	input	signal	Any	No	No	Yes
Cout:Cout:Cout	Metal1 pin	0.1	0.1	P	output	signal	Any	No	No	Yes
S:S:S	Metal1 pin	0.1	0.1	P	output	signal	Any	No	No	Yes
VDD:VDD:VDD	Metal3 pin	0.08	0.08	P	inputOutput	power	Any	No	No	Yes
VSS:VSS:VSS	Metal4 pin	0.08	0.08	P	inputOutput	ground	Any	No	No	Yes

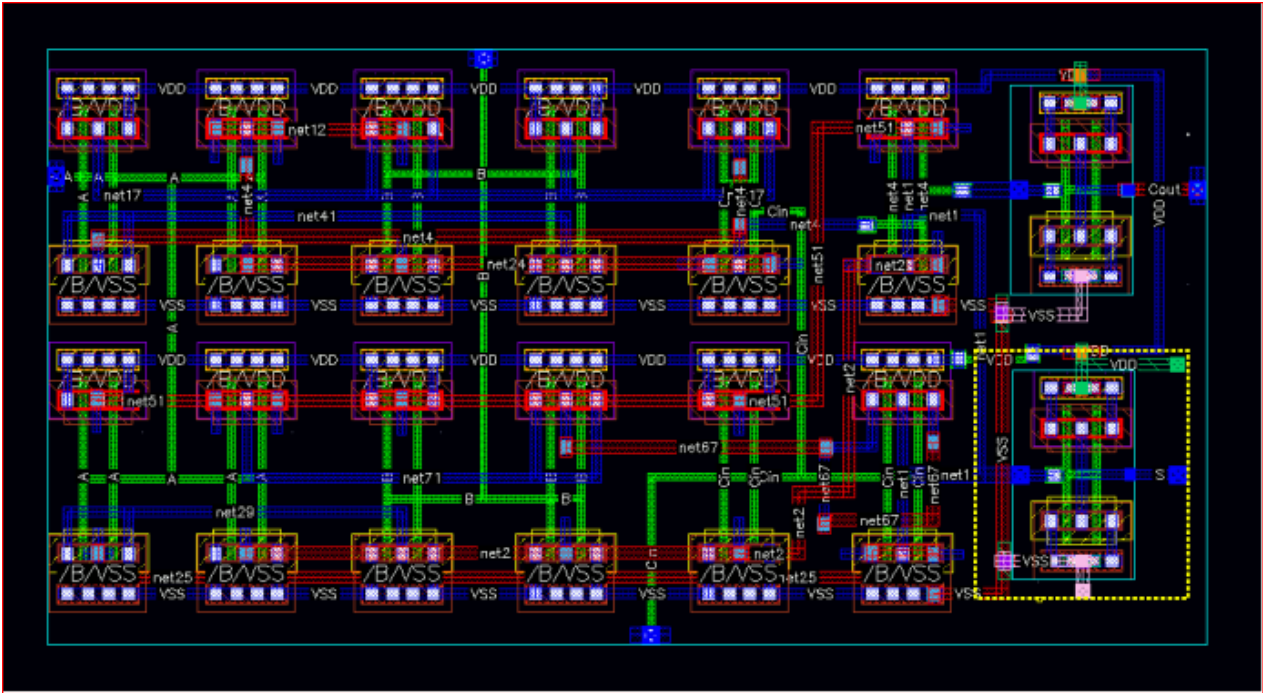
Cách sửa: sử dụng labels creation để tạo label cho các pin.



Kết quả LVS:



Layout cuối cùng:



1.3 Question

➤ Due to the topology shown in **Figure 3**, why do people implement the PDN of the first stage using the equation? $C_{out} = (A + B)C_{in} + AB$ instead of $C_{out} = AB + AC_{in} + BC_{in}$?

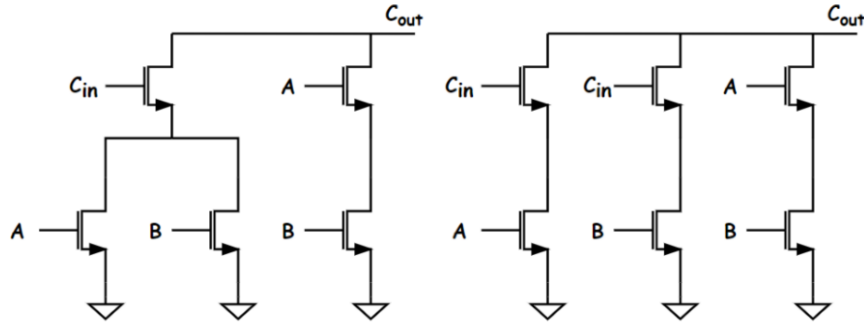
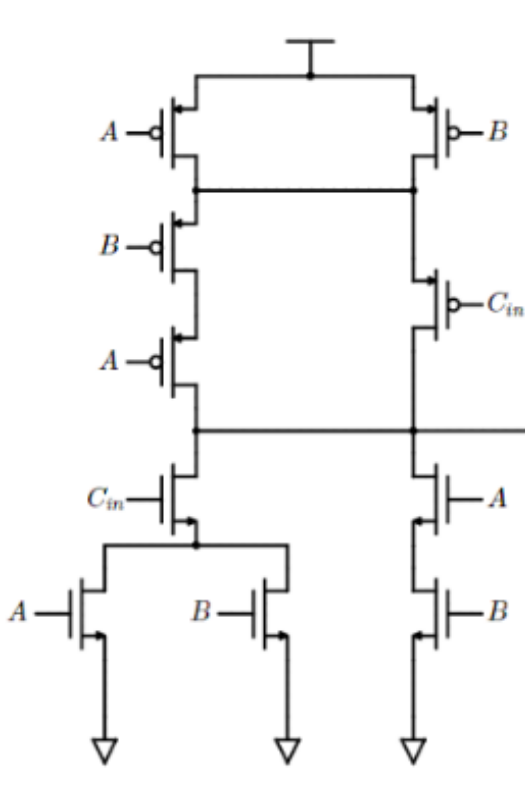
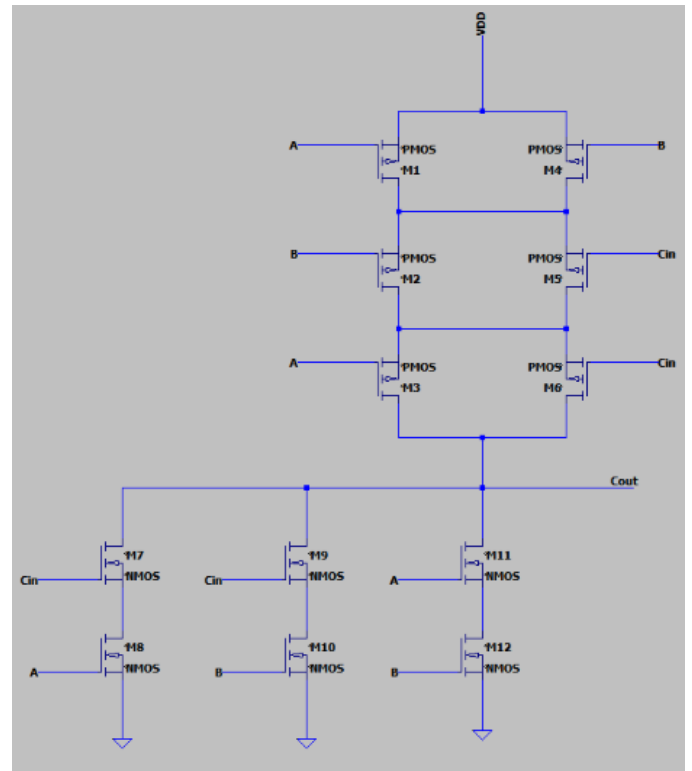


Figure 6 The PDN presents (left) $C_{out} = (A + B)C_{in} + AB$ (right) $C_{out} = AB + AC_{in} + BC_{in}$.

Hàm $C_{out} = (A + B)C_{in} + AB$ (tạm gọi là A) được dùng cho tầng 1 của 28T fulladder trên thay vì $C_{out} = AB + AC_{in} + BC_{in}$ (tạm gọi là B) là do:

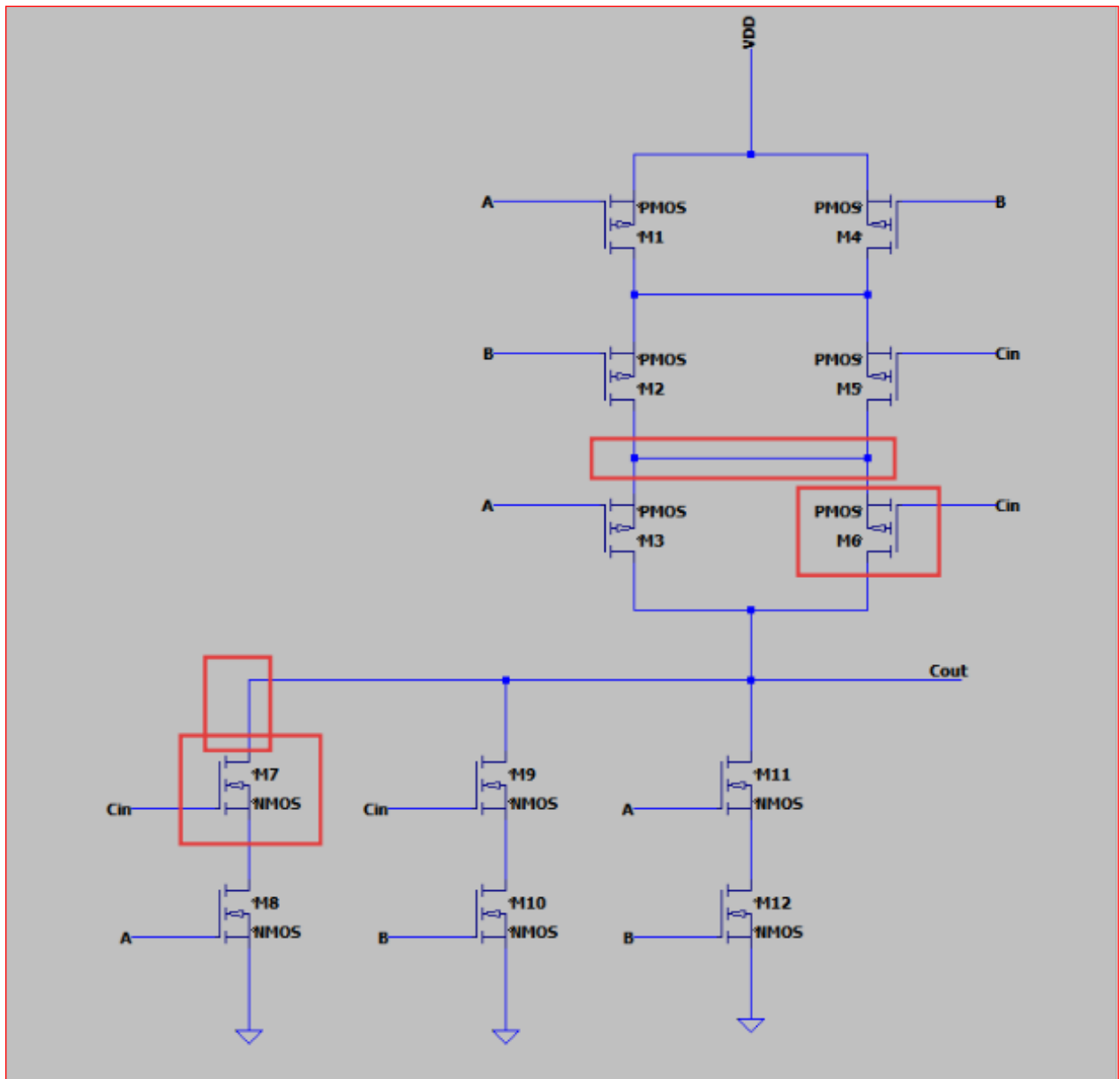


(a) Mạch hàm A



(b) Mạch hàm B

Ta thấy, mạch hàm B làm phát sinh thêm 2 MOSFET là M6 và M7 và 2 đường dây: từ Cout đến cực D M7 + đường dây nối các FET M2 M3 M5 M6:

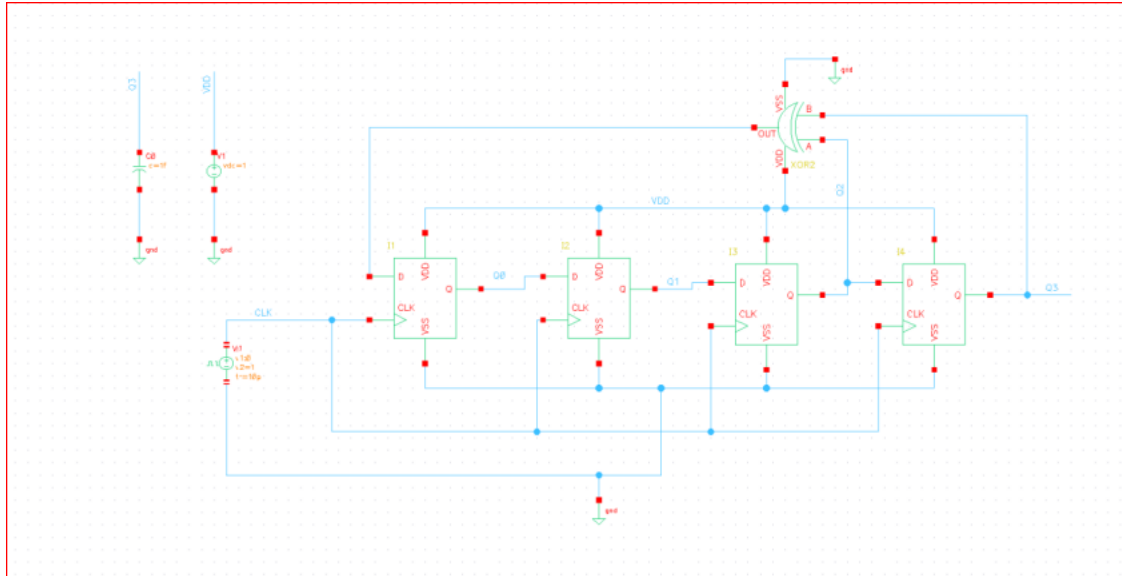


Hình 31: Các thành phần phát sinh

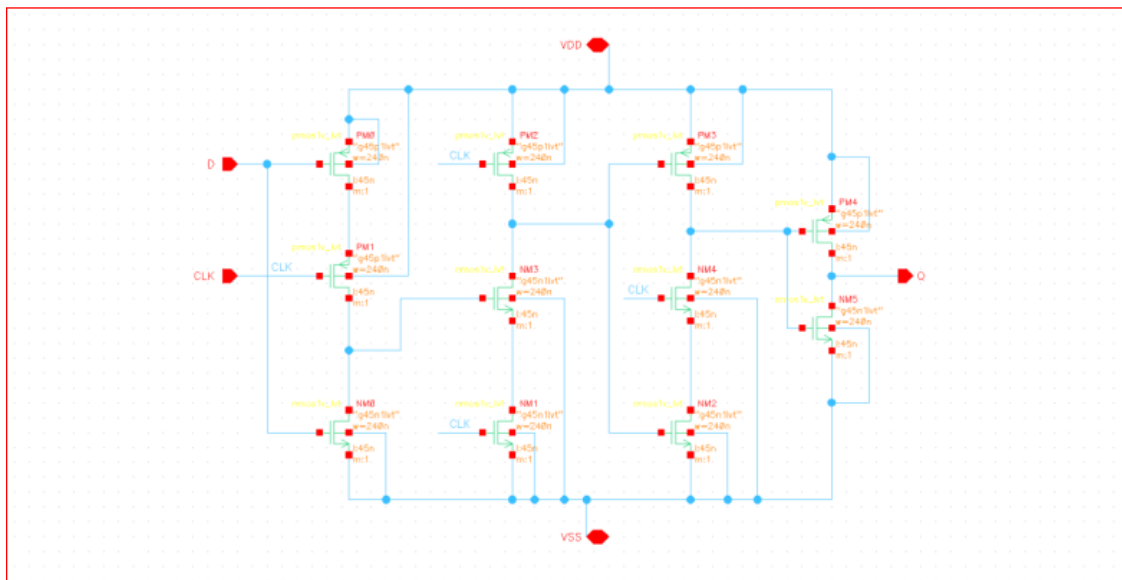
Tất cả các điểm trên làm tăng độ phức tạp của layout thực tế, qua đó làm tăng số lượng tụ kí sinh trên đường dây và các MOSFET phát sinh (M6, M7)

2. Experiment 2: Design a sequential circuit - A PRBS generator.

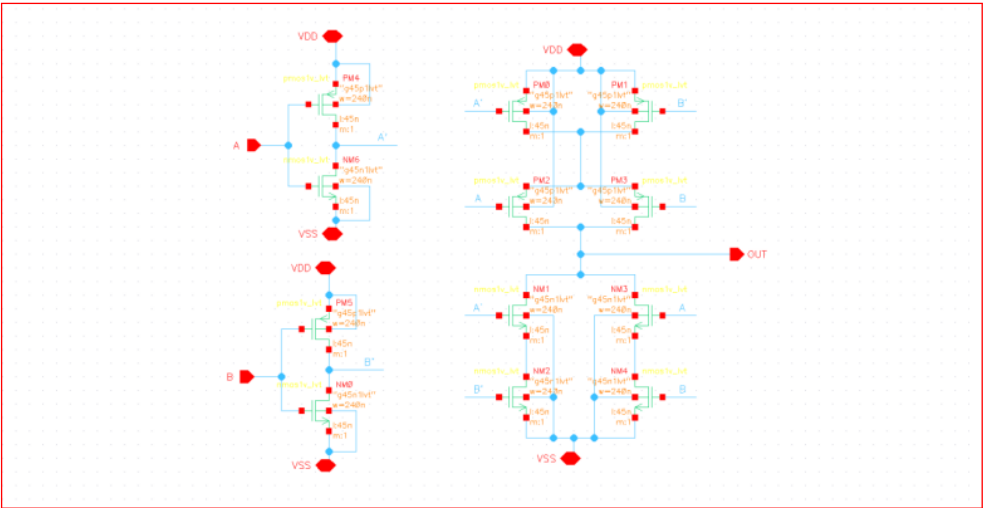
2.1 Schematic và testbench thiết kế bộ PRBS



Hình 32: Sơ đồ mạch thiết kế bộ PRBS



Hình 33: Sơ đồ mạch D_FF sử dụng để thiết kế bộ PRBS

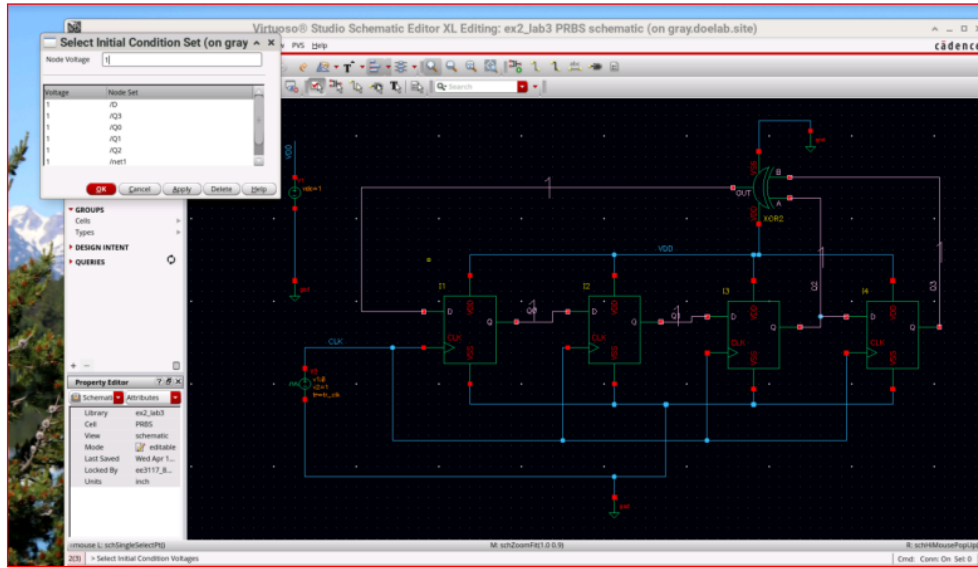


Hình 34: Sơ đồ mạch cổng XOR2 sử dụng trong thiết kế bộ PRBS



Hình 35: Testcase đưa vào nguồn vpulse CLK của bộ PRBS

Thiết lập điều kiện ban đầu (Initial Condition) cho các D_FF, sử dụng phương pháp setup có sẵn của tool như hình sau:



Hình 36: Chọn điều kiện đầu tổ hợp 1111 để bộ PRBS hoạt động

2.2 Thực hiện tính toán sơ bộ để khoanh vùng Fmax

Để tìm Fmax, ta cần tính thời gian tối thiểu để dữ liệu chạy qua tất cả các khối sao cho vẫn thấp hơn thời gian của một chu kỳ xung clk: $T_{clk} \geq t_{cq} + t_{setup} + t_{logic_max}$.

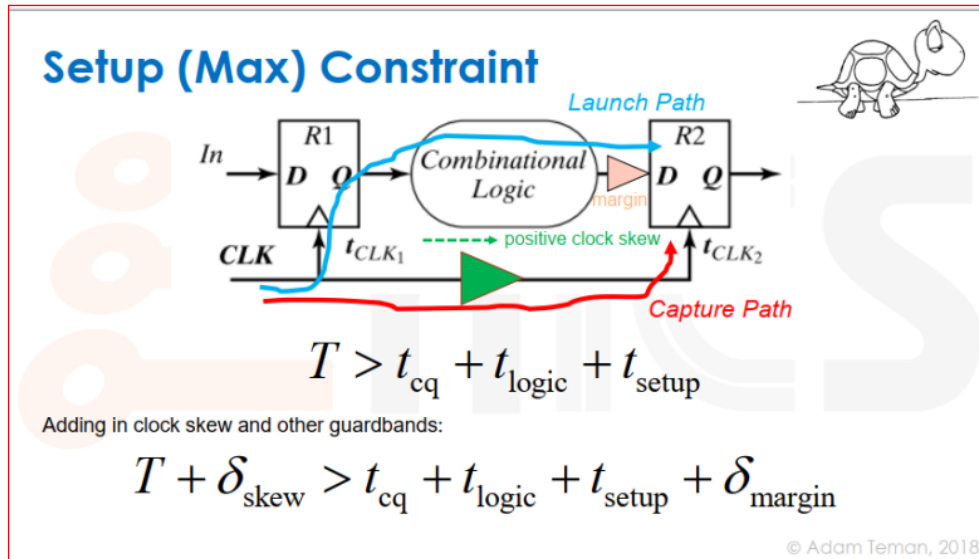
Tức ta phải đi tìm đường critical path (hay là đường dẫn tới hạn) là đường dẫn dữ liệu, tín hiệu có tổng thời gian trễ (delay) lớn nhất trong toàn bộ thiết kế. Nhìn vào schematic ở hình 32, ta có hai nhóm đường đi:

- Nhóm 1: (Đường dịch chuyển - Shift Path): Từ Q0 D của DFF2; từ Q1 D của DFF3; từ Q2 D của DFF4. $\Rightarrow$ Thời gian trễ ở đường này bằng $t_{cq} + t_{setup}$ (vì không có cổng logic nào ở giữa)
- Nhóm 2: (Đường phản hồi - Feedback path): Từ Q2 và Q3 chạy qua cổng XOR, rồi vòng về ngõ vào D của DFF1. $\Rightarrow$ Thời gian trễ ở đường này bằng $t_{cq} + t_{setup} + t_{logic}$

Đường phản hồi ở nhóm 2 tốn nhiều thời gian vì nó phải gánh thêm độ trễ lan truyền (t_{logic} hay t_{pd}) của cổng XOR. Đây chính là đường Critical Path quyết định giới hạn tốc độ của toàn mạch. Công thức tính tần số F_{max} :

$$F_{max} = 1/T_{clk_min}$$

(“min” trong công thức T_{clk_min} có nghĩa là thời gian tối thiểu cần thiết, để đảm bảo dữ liệu đi qua có thể đến kịp trong một chu kỳ clk tiếp theo).



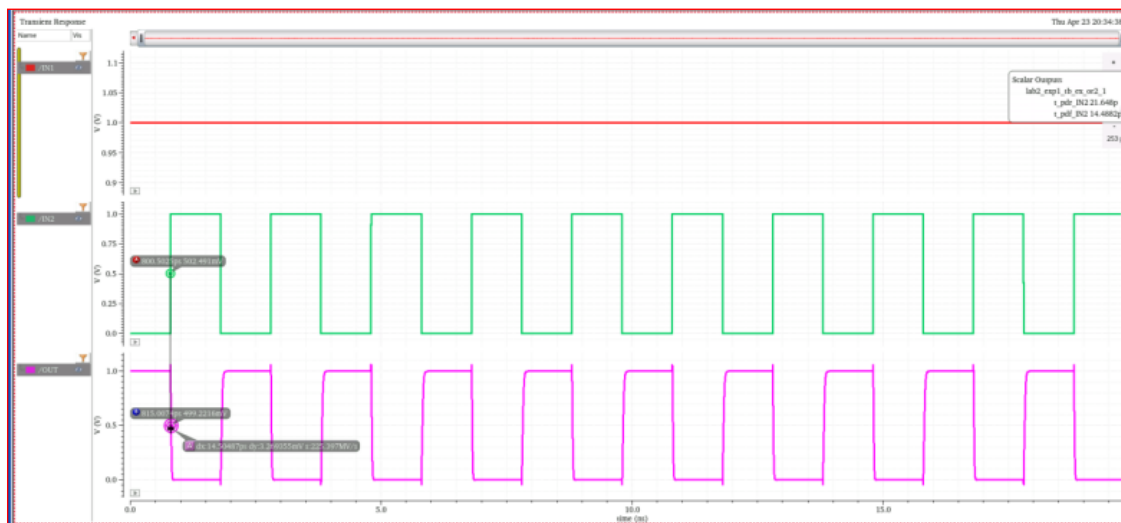
Hình 37: Hình minh họa về Concept đường đi dữ liệu

2.2.1 Đo lại t_{logic_max} (IN1, IN2) của cổng XOR2

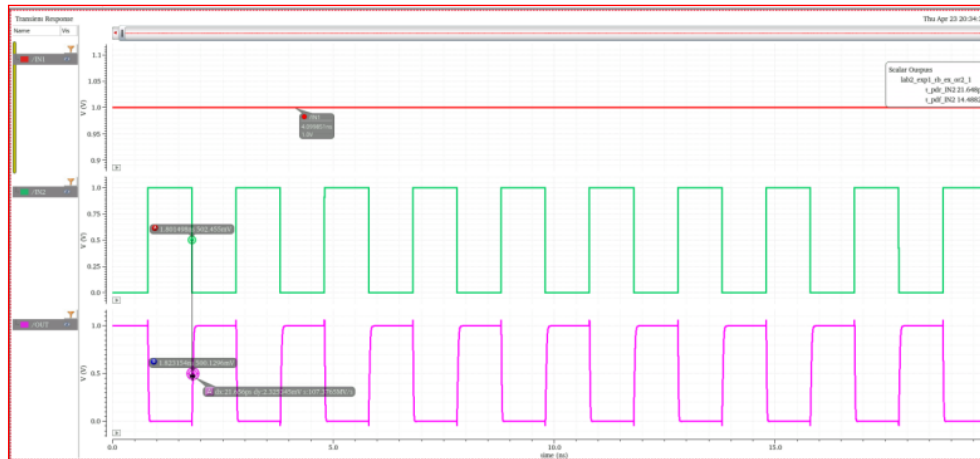
Kết quả đo lại t_{pdf} và t_{pdr} của hai ngõ vào IN1 và IN2 của hai trường hợp khi IN1 = 1 và khi IN1 = 0 và ngõ ra còn lại cho chạy so với ngõ ra OUT, làm tương tự như vậy với trường hợp ngược lại, ta thu được các kết quả sau:

	IN1=1V, IN2 vs OUT	IN1=0V, IN2 vs OUT	IN2=1V IN1 vs OUT	IN2=0V IN1 vs OUT
T_pdr	21.65p	33.2p	15.06p	30.83p
T_pdf	14.49p	26.31p	12.32p	23.71p
T_pd_aver	18.07p	29.755p	13.69p	27.27p
T_pd MAX	29.755p			

Một số hình ảnh minh họa một vài trường hợp đo t_{pd} :



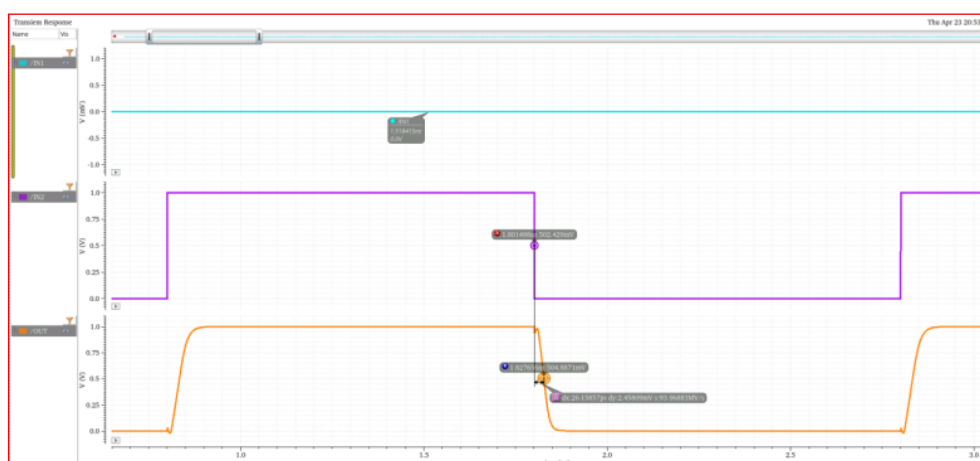
Hình 38: Đo t_{pdf} khi IN1 = 1V, IN2 vs OUT



Hình 39: Đo t_{pdr} khi $IN1 = 1V$, $IN2$ vs OUT



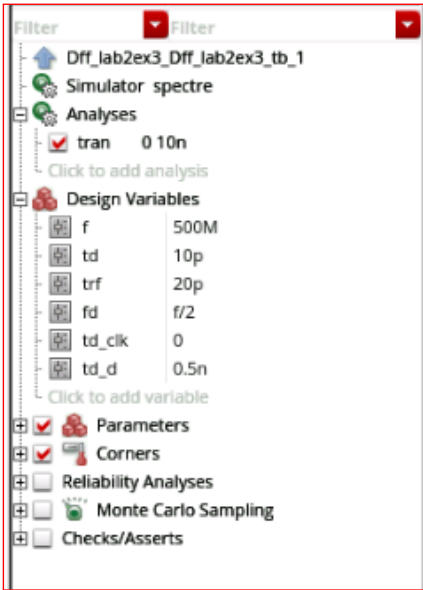
Hình 40: Đo t_{pdr} khi $IN1 = 0V$, $IN2$ vs OUT



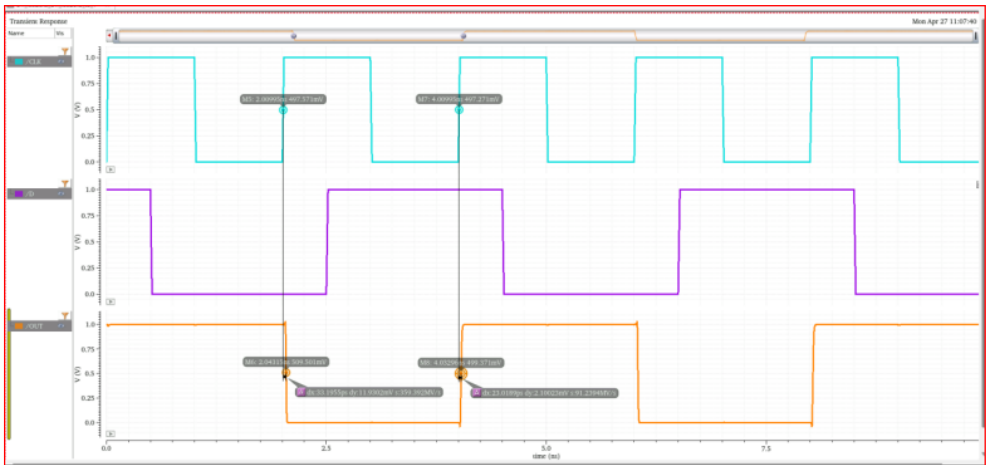
Hình 41: Đo t_{pdf} khi $IN1 = 0V$, $IN2$ vs OUT

2.2.2 Đo lại t_{cq} và t_{setup} của D_FF

Đo lại t_{cq} của D_FF



Hình 42: Testcase đo chung cho t_{cq} và t_{setup}



Hình 43: Mô phỏng đúng chức năng và thực hiện đo t_{cq} (thời gian để ngõ ra đáp ứng kể từ xung clock - từ clk đến Q)

t_cqf	expr	delay(7wf1 VT("/CLK")) ?value1 0.5 ?edge1 *ris...	22.98p	☒	☐	☐
t_cqr	expr	delay(7wf1 VT("/CLK")) ?value1 0.5 ?edge1 *ris...	33.26p	☒	☐	☐

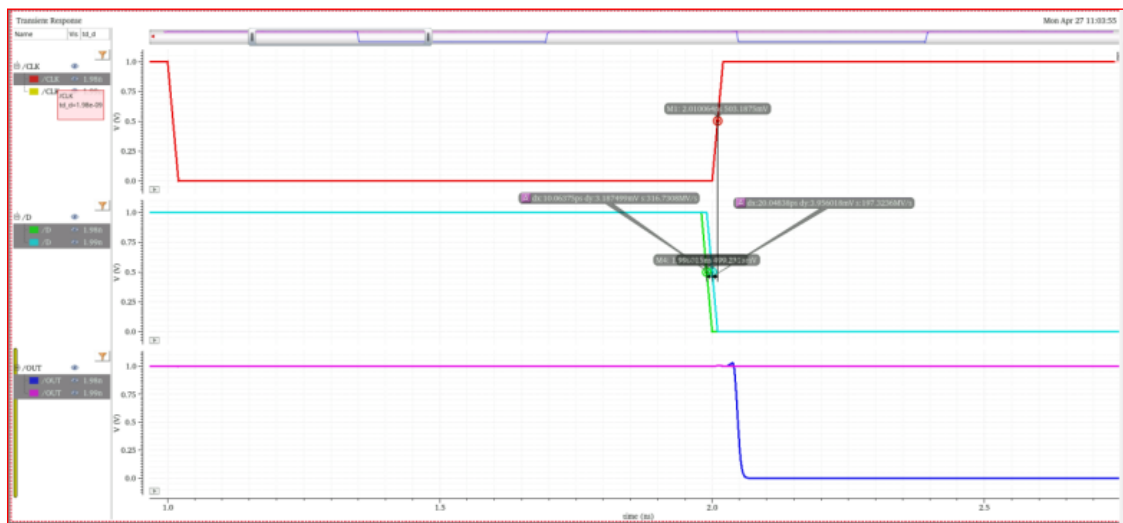
Hình 44: Kết quả đo bằng hàm

Nhóm tổng hợp kết quả lại như sau:

T _{cqf}	33.26p
T _{cqr}	22.98p
T _{cq_MAX}	33.26p

Đo lại t_{setup} của D_FF

- Thực hiện đo t_{setup} bằng phương pháp shift ngõ vào càng gần cạnh xung clk sao cho đến khi ngõ ra bị vi phạm, không xuất đúng logic nữa. Biến shift là td_d
- Lần lượt đo thô cài đặt biến td_d chạy trong khoảng: 0.5n:50p:2n (30 giá trị)
- Shift dần thu được ranh giới ngõ ra OUT PASS và FAIL ở 1.95 và 2.0.
- Tiếp tục thu hẹp phạm vi cài biến td_d chạy từ: 1.95n:0.1n:2.0n
- Thu được kết quả ranh giới chính xác ngõ ra PASS/FAIL ở 1.98n và 1.99n



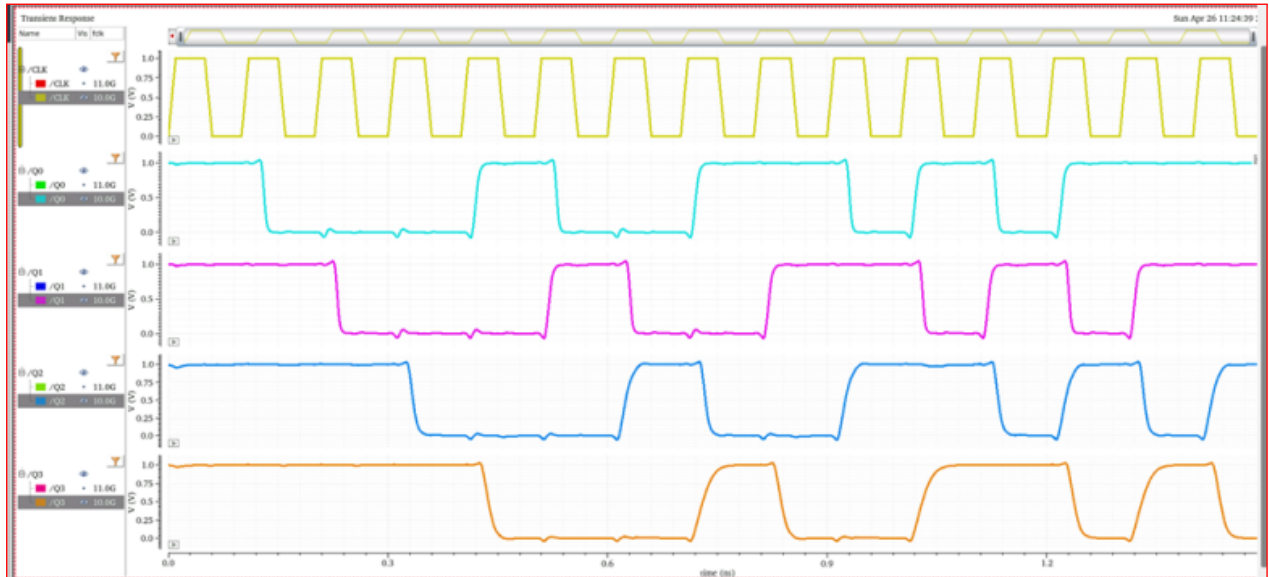
Hình 45: Kết quả đo t_{setup}

Td_d	1.98n	1.99n
T_clk đến D	10.06375p	20.04838p
Ngõ ra OUT (Q)	PASS	FAIL
T_setup_aver	$\frac{10.06375 + 20.04838}{2} \approx 15.1p$	

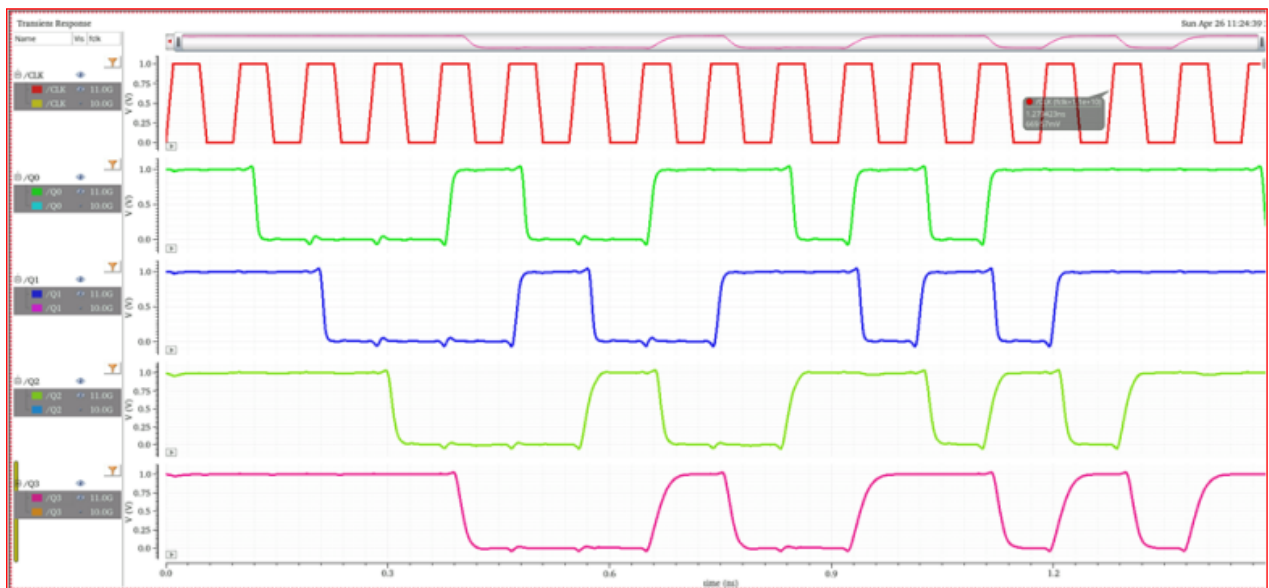
Tóm lại: $T_{\text{clk_min}} = t_{\text{cq_MAX}} + t_{\text{setup_aver}} + t_{\text{logic_MAX}} = 33.26ps + 15.1ps + 29.755ps = 78.115ps \Rightarrow F_{\text{MAX}} = 1/T_{\text{clk_min}} \approx 12.8GHz$

2.3 Thực hiện đo quét tần số xung quanh F_{MAX} lý thuyết trên bộ PRBS:

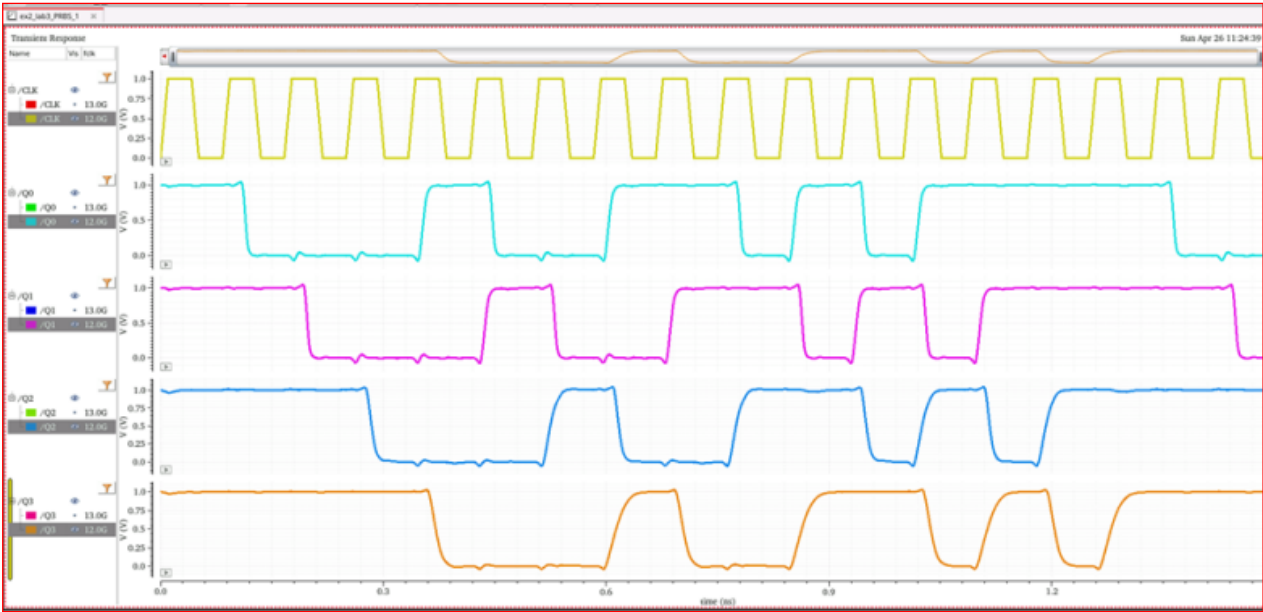
2.3.1 Hình ảnh mô phỏng quét tần số từ 10G đến 20G bước nhảy 1G



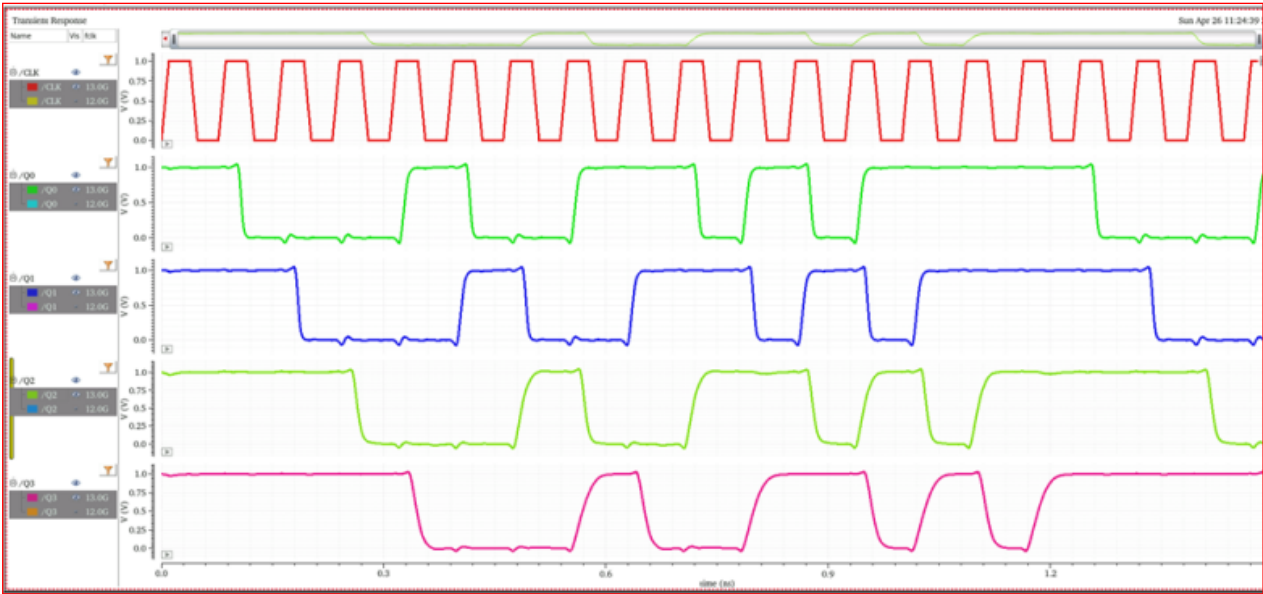
Hình 46: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 10G$



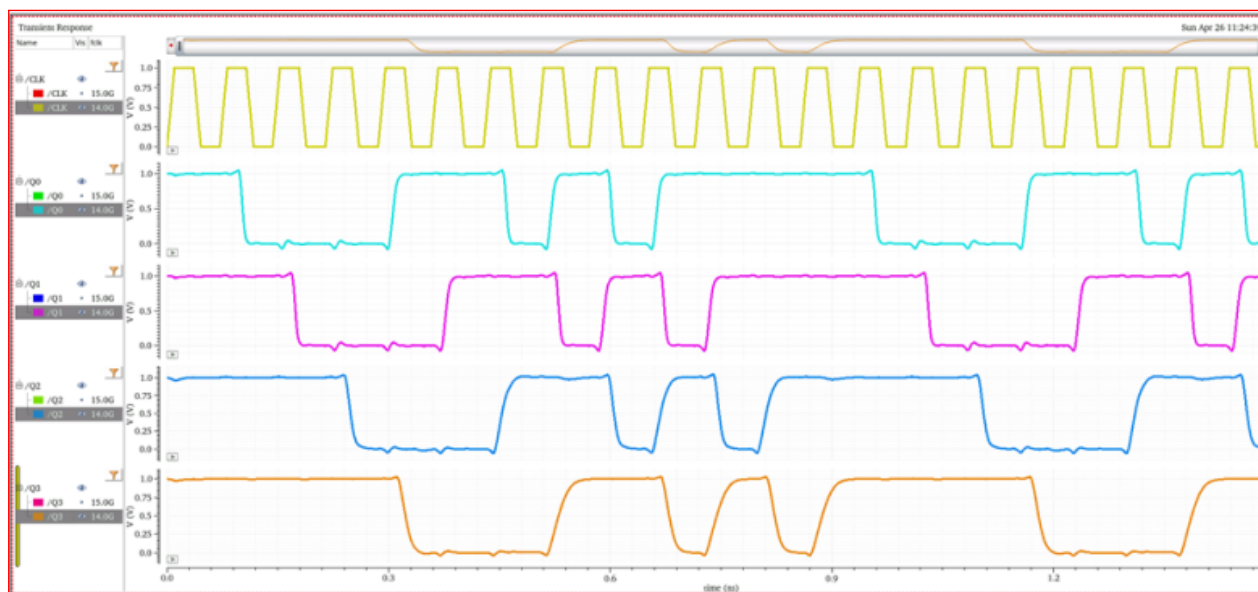
Hình 47: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 11G$



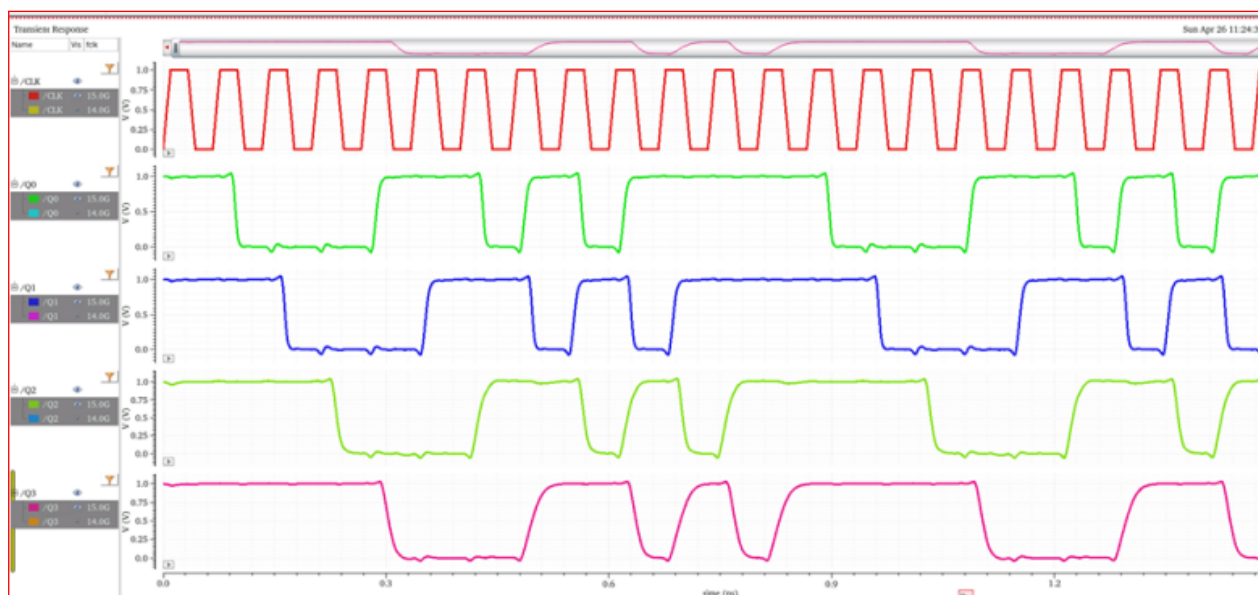
Hình 48: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 12G$



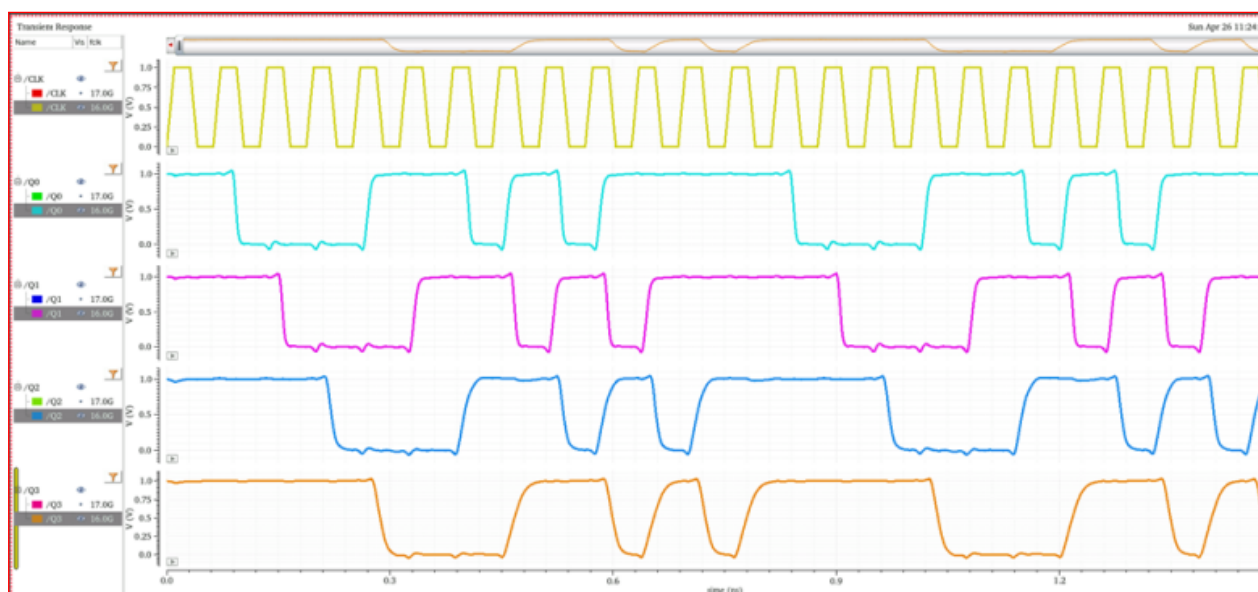
Hình 49: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 13G$



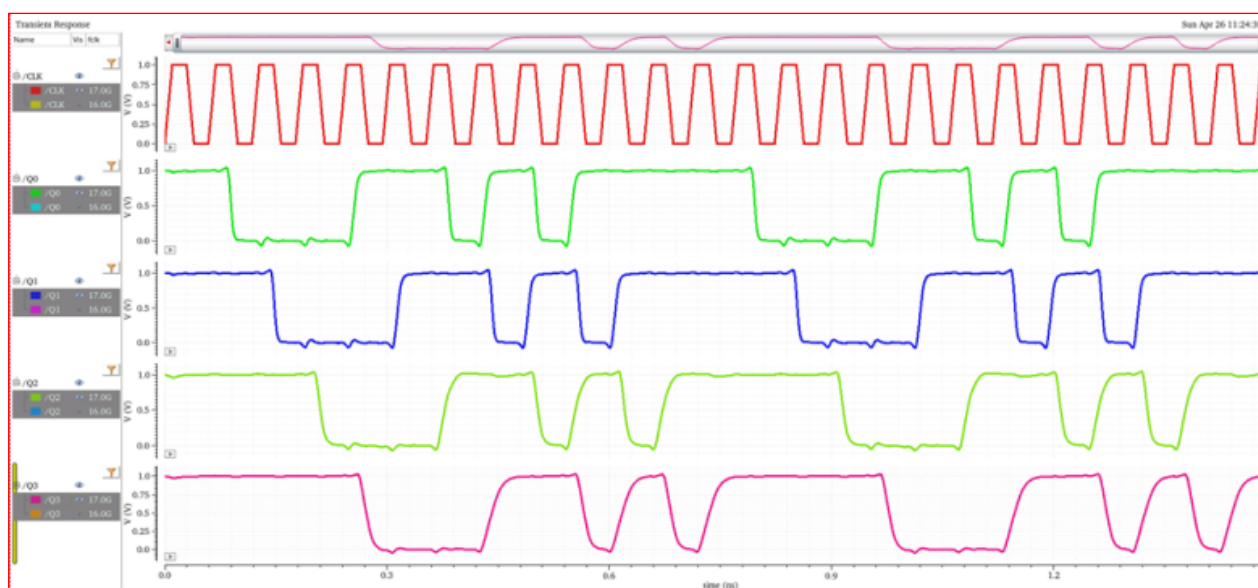
Hình 50: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 14G$



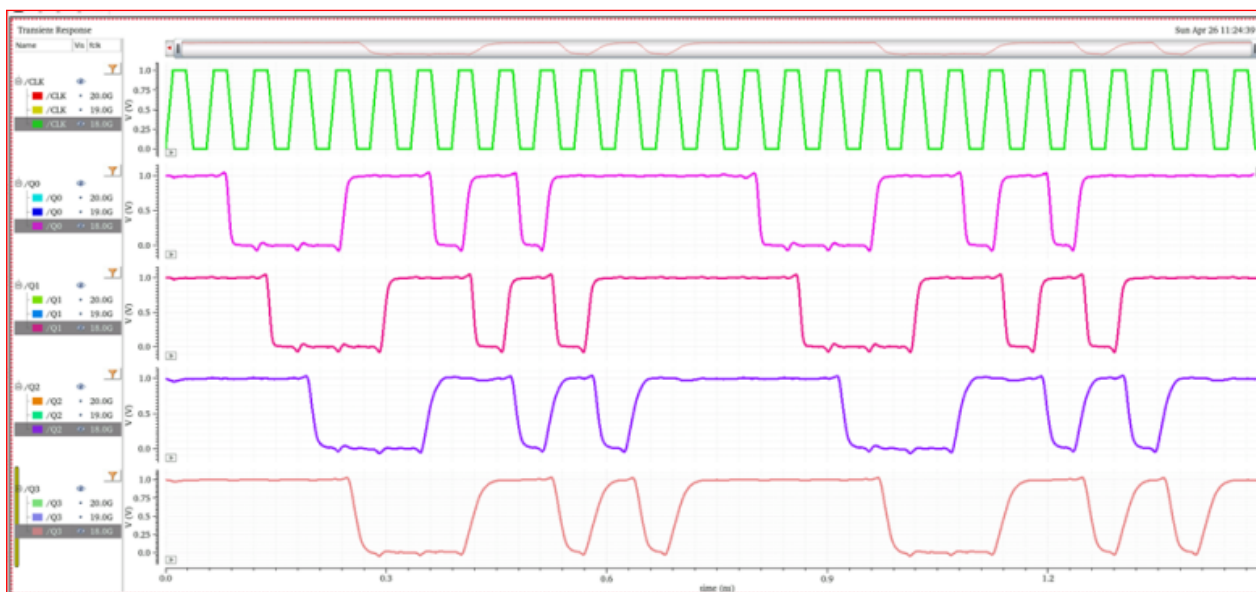
Hình 51: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 15G$



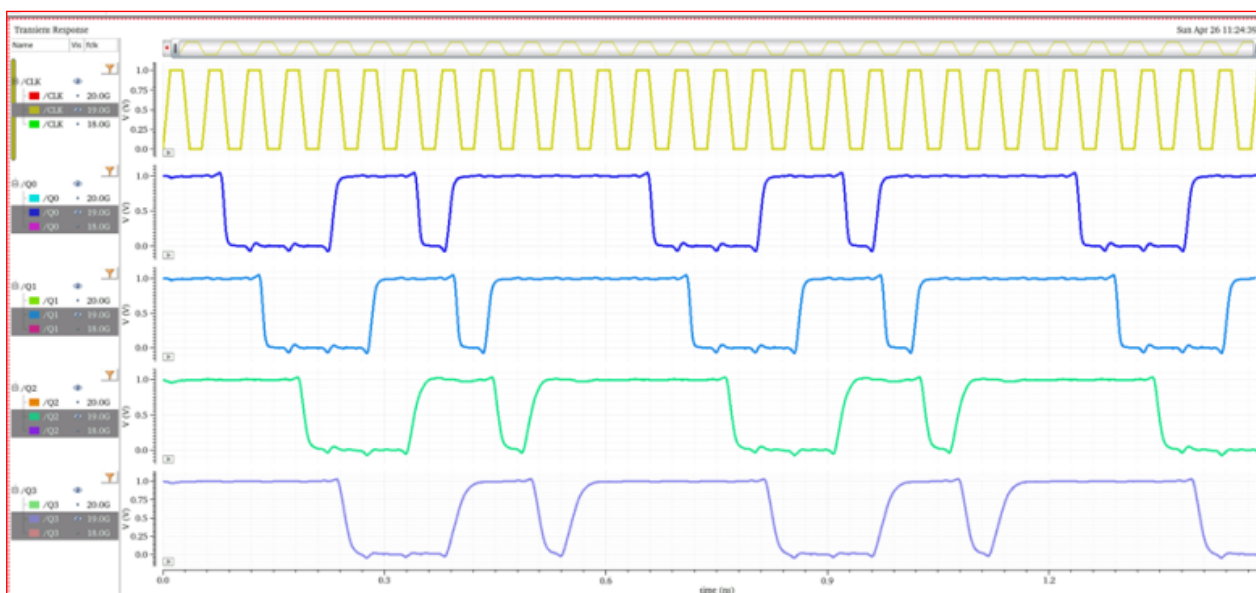
Hình 52: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 16G$



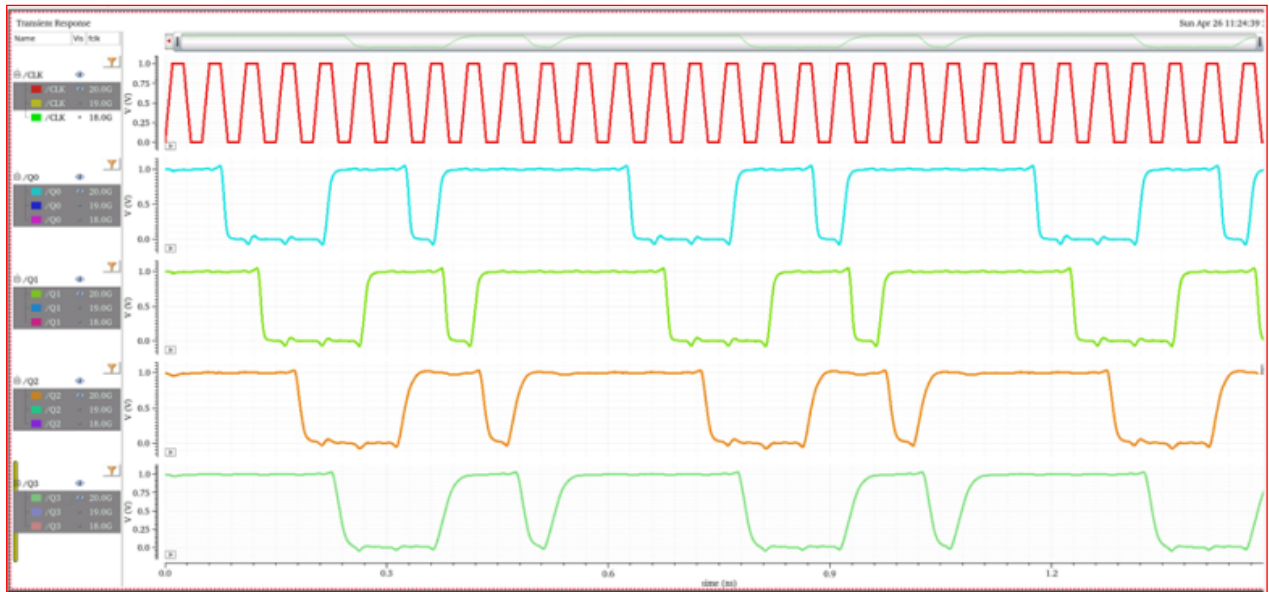
Hình 53: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 17G$



Hình 54: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 18G$



Hình 55: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 19G$



Hình 56: Đáp ứng ngõ ra của bộ PRBS khi tần số $F = 20\text{GHz}$

2.3.2 Kết luận về tần số hoạt động tối đa ($F_{\max}$) của bộ tạo mã PRBS-4bit

Kết quả xác định $F_{\max}$: Qua quá trình quét tần số trên mô phỏng, tần số hoạt động tối đa ổn định của mạch được xác định là: $F_{\max} = 13\text{ GHz}$. Tại mức 13 GHz:

- Chức năng: Mạch đếm đúng 100% tổ hợp chuỗi dữ liệu.
- Tín hiệu: Dạng sóng hiển thị rõ nét, vuông vức. Biên độ điện áp nạp xả đầy đủ, đạt chuẩn mức logic an toàn (mức 1 lên sát 1.0V, mức 0 về sát 0V).

Đánh giá dải tần số cao hơn (14 GHz đến 20 GHz)

- Tại 14 GHz (Ngưỡng giới hạn): Mạch vẫn chạy đúng chức năng, nhưng do tốc độ quá nhanh, tín hiệu bắt đầu bị bóp nhọn. Mức điện áp chỉ vừa vặn chạm ngưỡng nhận diện tối thiểu. Đây là điểm rủi ro, không nên chọn làm giới hạn hoạt động.
- Từ 18 GHz đến 20 GHz (Không đạt yêu cầu): Ở tần số này, mạch bị thiếu thời gian nghiêm trọng. Khi xuất ra một nhịp bit ngắn, tín hiệu chưa kịp đi lên mức 1 thì đã bị ép quay đầu đi xuống, gây ra hiện tượng sụt giảm biên độ nặng nề. Dù mạch vẫn đang cố nhảy số, nhưng các cổng logic ở tầng sau sẽ không thể đọc hiểu được tín hiệu lấp lửng này.

Kết quả kiểm tra, chạy thử code phần mềm Python của bộ tạo PRBS

```

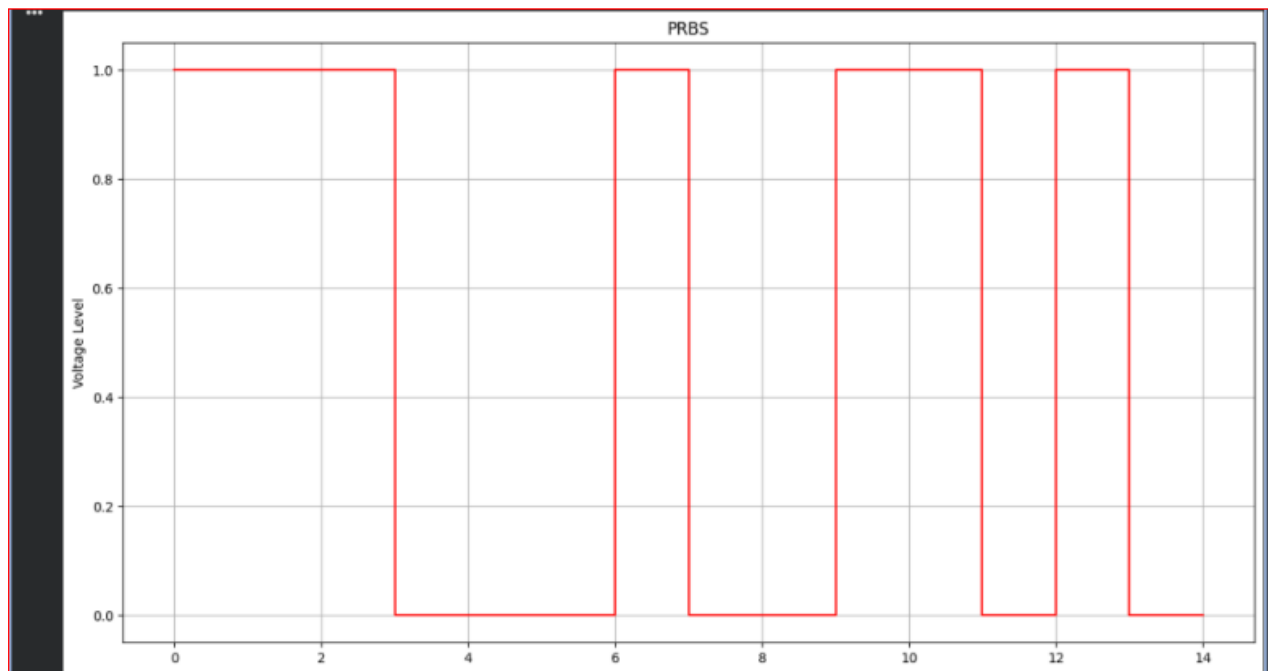
# -*- coding: utf-8 -*-
"""
Created on Tue Oct 10 22:48:56 2023
@author: nelsonn
"""

import matplotlib.pyplot as plt
#Generate binary string using PRBS
#number of bits you want
n = int(input('Number of bit of PRBS you want: '))
#random a number from 0 <- x < 2^n -1
state = int(input('Initial state: '), 2)
output_PRBS = []
for i in range(2**n - 1):
    print(state & 1, end = ' ')
    output = state & 1
    output_PRBS.append(output)
    newbit = (state ^ (state >> 1)) & 1
    state = ((state >> 1) | (newbit << (n-1)))
print('A randomized bit stream: ', output_PRBS)
print('Length of randomized bit stream = ', len(output_PRBS))
#Plotting
plt.tight_layout()
plt.figure(figsize=(15, 8))
plt.step(range(len(output_PRBS)), output_PRBS, drawstyle='steps', color='r')
plt.title('PRBS')
plt.ylabel('Voltage Level')
plt.grid(True)
plt.show()

```

Number of bit of PRBS you want: 4
 Initial state: 1111
 111100010011010A randomized bit stream: [1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0]
 Length of randomized bit stream = 15

Hình 57: Kết quả chạy code Python bộ tạo mã PRBS-4bit



Hình 58: Dạng sóng mô phỏng bộ PRBS-4bit

3. Experiment 3: Design a VerilogA model.

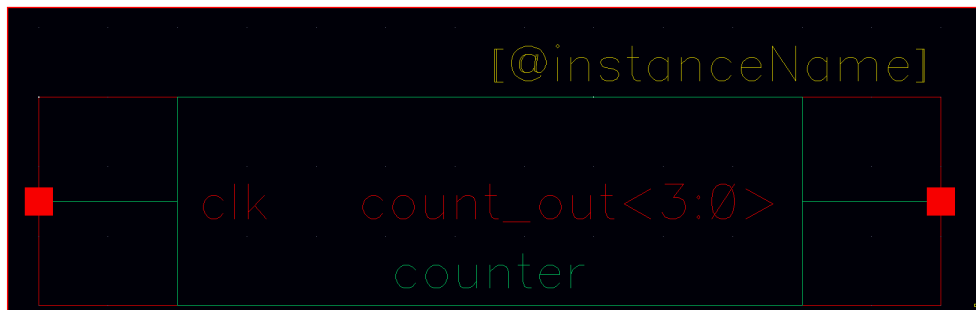
Đoạn code và symbol cho khối 4 bit counter up bằng VerilogA:

```

1 // VerilogA for lab3_exp3, counter, veriloga
2 `include "constants.vams"
3 `include "disciplines.vams"
4
5 module counter(clk, count_out);
6   input clk;
7   output [3:0] count_out;
8   electrical clk;
9   electrical [3:0] count_out;
10
11   parameter real vss = 0;
12   parameter real vdd = 1;
13   parameter real vth = 0.5;
14   parameter real trise = 0.5n;
15   parameter real tfall = 0.5n;
16   parameter real tdelay = 0.5n;
17
18   integer count;
19   analog begin
20     @(initial_step) count = 0;
21     @(cross(V(clk) - vth, 1)) begin
22       count = (count + 1) % 16;
23     end
24
25     V(count_out[0]) <+ transition((count & 1) ? vdd : vss, trise, tfall, tdelay);
26     V(count_out[1]) <+ transition((count & 2) ? vdd : vss, trise, tfall, tdelay);
27     V(count_out[2]) <+ transition((count & 4) ? vdd : vss, trise, tfall, tdelay);
28     V(count_out[3]) <+ transition((count & 8) ? vdd : vss, trise, tfall, tdelay);
29   end
30 endmodule

```

Hình 59: Code VerilogA khối 4 bit counter-up



Hình 60: Symbol khối 4 bit counter-up

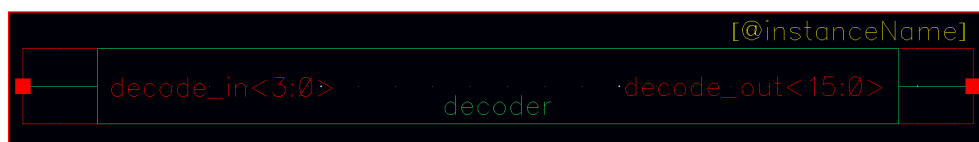
Đoạn code và symbol cho khối 4 bit decoder bằng VerilogA:

```

1 // VerilogA for lab3_exp3, decoder, veriloga
2
3 `include "constants.vams"
4 `include "disciplines.vams"
5 module decoder(decode_in, decode_out);
6     input [3:0] decode_in;
7     output [15:0] decode_out;
8     electrical [3:0] decode_in;
9     electrical [15:0] decode_out;
10
11     parameter real vdd = 1;
12     parameter real vss = 0;
13     parameter real vth = 0.5;
14     parameter real tdelay = 0.5n;
15     parameter real trise = 0.5n;
16     parameter real tfall = 0.5n;
17
18     integer state;
19     analog begin
20         state = 0;
21         if (V(decode_in[0]) > vth) state = state + 1;
22         if (V(decode_in[1]) > vth) state = state + 2;
23         if (V(decode_in[2]) > vth) state = state + 4;
24         if (V(decode_in[3]) > vth) state = state + 8;
25
26         V(decode_out[0]) <+ transition((state == 0) ? vdd : vss, tdelay, trise, tfall);
27         V(decode_out[1]) <+ transition((state == 1) ? vdd : vss, tdelay, trise, tfall);
28         V(decode_out[2]) <+ transition((state == 2) ? vdd : vss, tdelay, trise, tfall);
29         V(decode_out[3]) <+ transition((state == 3) ? vdd : vss, tdelay, trise, tfall);
30         V(decode_out[4]) <+ transition((state == 4) ? vdd : vss, tdelay, trise, tfall);
31         V(decode_out[5]) <+ transition((state == 5) ? vdd : vss, tdelay, trise, tfall);
32         V(decode_out[6]) <+ transition((state == 6) ? vdd : vss, tdelay, trise, tfall);
33         V(decode_out[7]) <+ transition((state == 7) ? vdd : vss, tdelay, trise, tfall);
34         V(decode_out[8]) <+ transition((state == 8) ? vdd : vss, tdelay, trise, tfall);
35         V(decode_out[9]) <+ transition((state == 9) ? vdd : vss, tdelay, trise, tfall);
36         V(decode_out[10]) <+ transition((state == 10) ? vdd : vss, tdelay, trise, tfall);
37         V(decode_out[11]) <+ transition((state == 11) ? vdd : vss, tdelay, trise, tfall);
38         V(decode_out[12]) <+ transition((state == 12) ? vdd : vss, tdelay, trise, tfall);
39         V(decode_out[13]) <+ transition((state == 13) ? vdd : vss, tdelay, trise, tfall);
40         V(decode_out[14]) <+ transition((state == 14) ? vdd : vss, tdelay, trise, tfall);
41         V(decode_out[15]) <+ transition((state == 15) ? vdd : vss, tdelay, trise, tfall);
42     end
43 endmodule
44

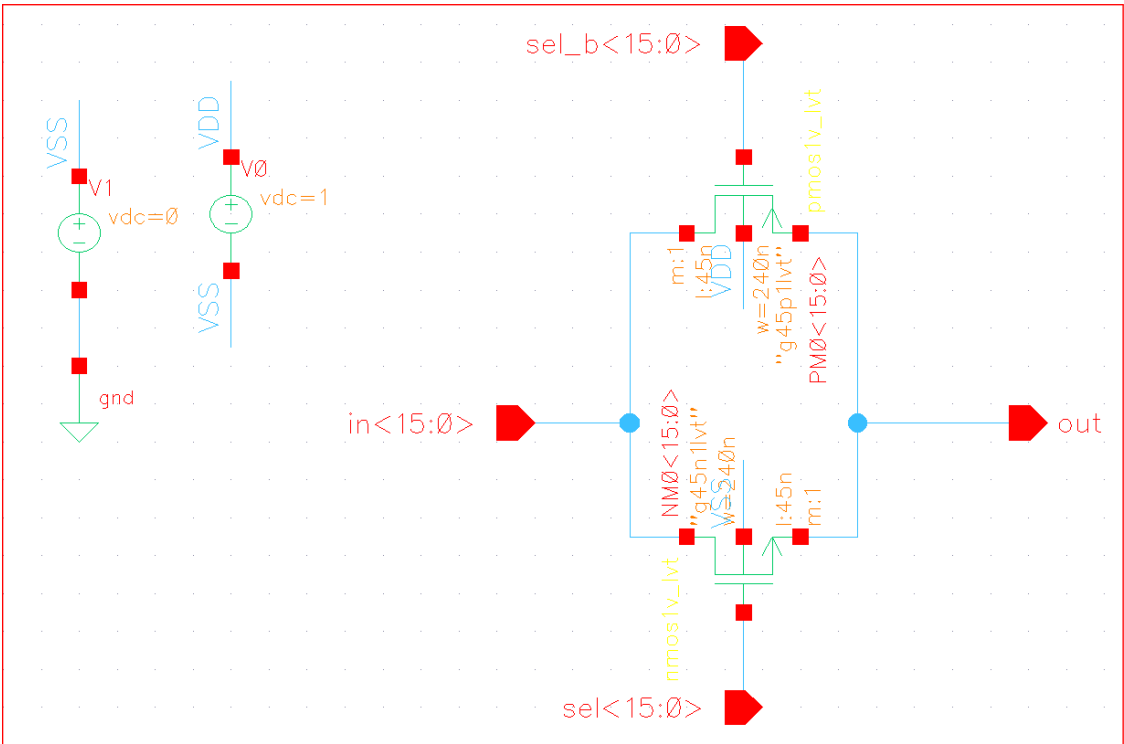
```

Hình 61: Code VerilogA khối 4 bit decoder

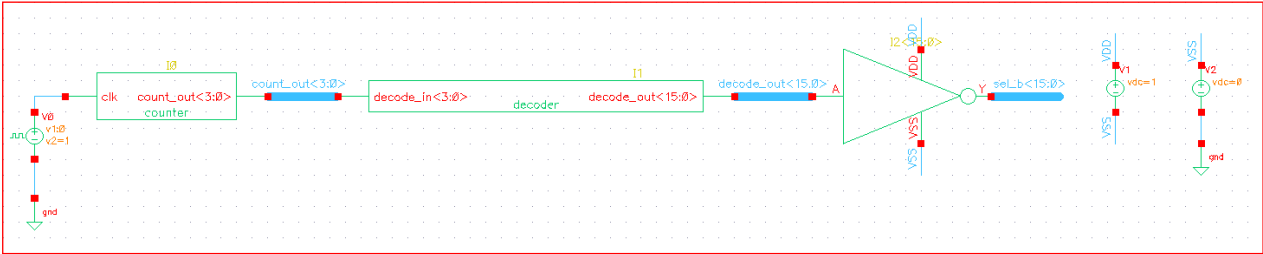


Hình 62: Symbol khối 4 bit decoder

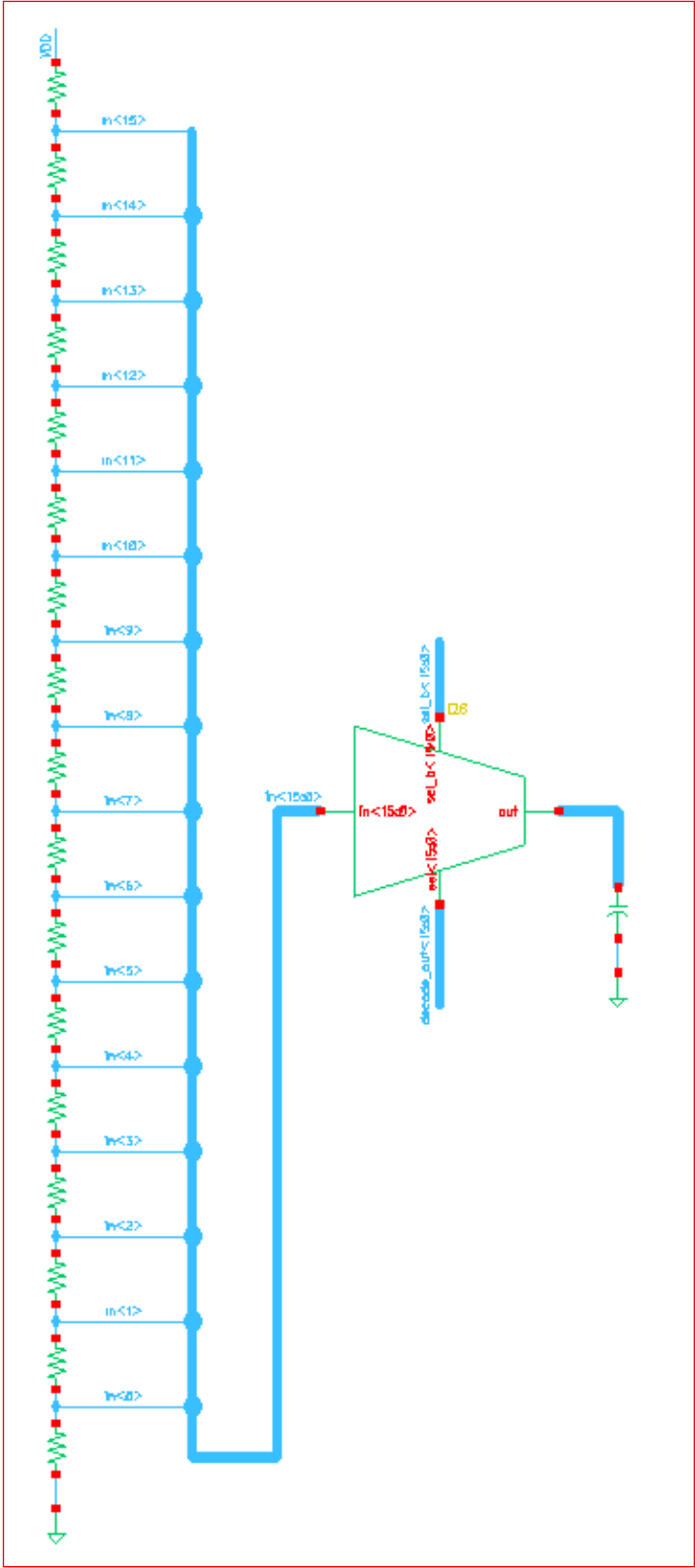
Mạch mux transmission gate:



Hình 63: Mux transmission gate

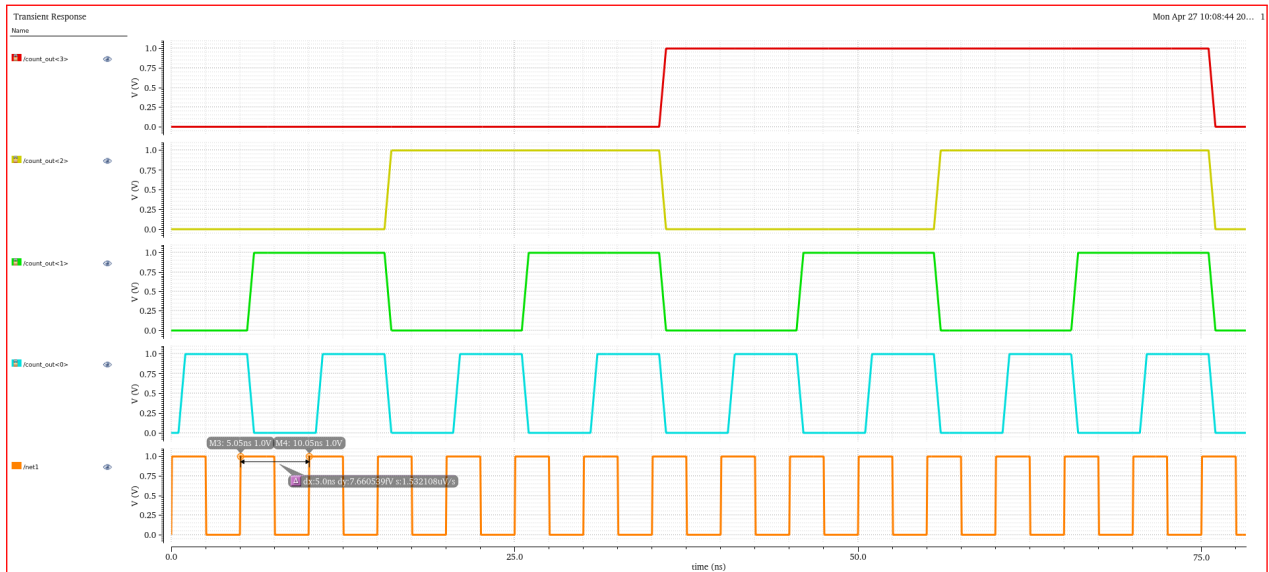


Hình 64: Khối verilogA trong mạch testbench



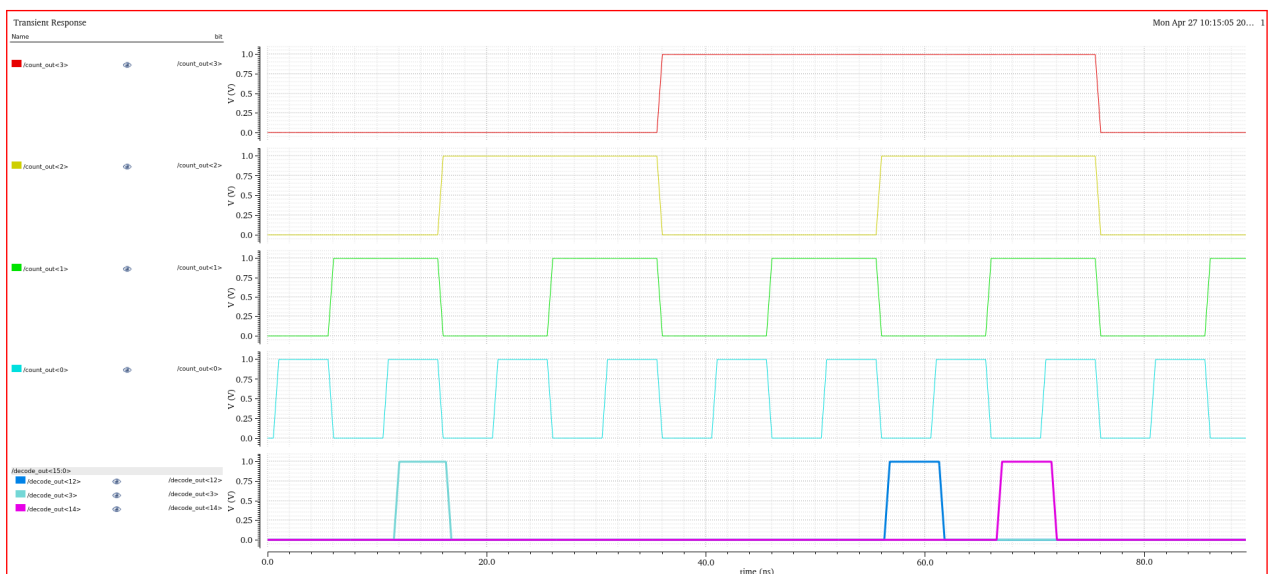
Hình 65: Khối resistor ladder và mux 16 sang 1 dùng transmission gate

Nhóm tiến hành kiểm chứng khối 4 bit counter up, kết quả ở hình bên dưới cho thấy mạch chạy đúng trình tự logic mà đề bài đặt ra:



Hình 66: Kết quả khối 4 bit counter up

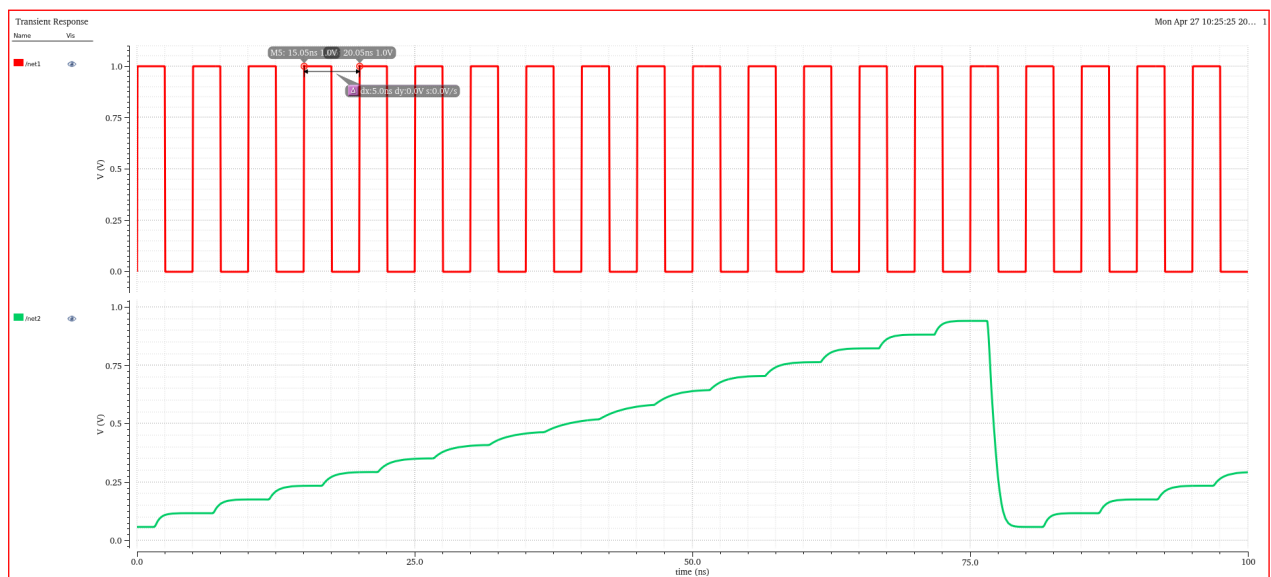
Nhóm tiến hành kiểm chứng khối 4 bit decoder, kết quả mô phỏng cho thấy mạch chạy đúng trình tự logic mà đề bài yêu cầu:



Hình 67: Kết quả khối 4 bit decoder

Ở đây nhóm lấy tương ứng 3 tổ hợp ngõ vào để kiểm chứng. Với tổ hợp ngõ vào 0010 thì ngõ ra tương ứng là bit thứ 3 của decode_out sẽ được kích lên 1, tương tự như vậy so với 2 ngõ ra còn lại.

Sau khi nhóm kiểm chứng 2 khối được viết bằng VerilogA chạy đúng với logic của đề bài yêu cầu, nhóm tiến hành kiểm chứng trên toàn bộ mạch.



Hình 68: Kết quả ngõ ra mux 16 sang 1

Kết quả hình trên cho thấy mạch chạy đúng trình tự logic. Với ngõ vào là xung clock 200Mhz thì ngõ ra tương ứng ở mux 16 sang 1 sẽ có dạng hình bậc thang tăng dần từ 58,82 mV cho tới 940,47 mV.